

Database development with PL/SQL INSY 8311



ADVENTIST UNIVERSITY
OF CENTRAL AFRICA

Instructor:

- Eric Maniraguha | ericmaniraguha2024@gmail.com | [LinkedIn Profile](#)



6h00 pm – 8h50 pm

- Monday A -G209
- Tuesday B-G209
- Wednesday E-G209
- Thursday F-G308

November 2024

Database development with PL/SQL

Reference reading



- [SQL Function](#)
- [PL/SQL Exception Handling](#)
- [What is Package in Oracle?](#)
- [Pipelined Table Functions](#)
- [Pipelined Table Functions](#)
- [What is PL/SQL? Explained](#)
- [SQL CASE Statement with Multiple Conditions](#)
- [Database Security](#)
- [7 Coding PL/SQL Procedures and Packages](#)

Lecture 08 – Procedures, Functions, and Packages

Procedures, Functions, and Packages Objectives



Understand PL/SQL Subprograms:

- Students will learn the fundamental differences between **procedures** and **functions** in PL/SQL, including when and how to use each.

Develop and Implement Procedures:

- Students will be able to define, create, and call **procedures** in PL/SQL to perform specific tasks within a database, including passing parameters and handling transactions.

Design and Use Functions:

- Students will understand the structure and purpose of **functions**, including how to create and use them in PL/SQL for returning computed values.
- They will also explore the practical use of functions within SQL queries and PL/SQL blocks.

Explore PL/SQL Packages:

- Students will learn the concept of **packages** in PL/SQL and understand the benefits of using them to group related procedures, functions, variables, and cursors.
- They will develop a package specification and body, implementing encapsulated, modular solutions for database applications.

Apply Subprograms for Modular Programming:

- By the end of the lecture, students will be able to organize and maintain their PL/SQL code efficiently by using **packages** for better modularity, reusability, and performance.

Execute and Debug PL/SQL Subprograms:

- Students will practice executing procedures and functions and gain hands-on experience in debugging and handling errors within PL/SQL subprograms.

Analyze the Benefits of Encapsulation and Reusability:

- Students will identify the benefits of encapsulating related code within packages to promote code reusability, security, and better performance in PL/SQL applications.

Key Points about Procedures in PL/SQL

1. Definition:

- A **procedure** is a subprogram that can accept parameters and perform actions. It is mainly used to execute operations such as modifying database records, performing calculations, or invoking other subprograms.

It consists of two parts: **specification** (starting with the keyword PROCEDURE) and **body** (beginning with IS or AS).

- Parameters are optional and can be included in the spec.



2. Procedure Body:

- The body has three parts:
 - **Declarative part** (optional): contains variable, constant, exception, and type declarations.
 - **Executable part**: contains the actual executable statements.
 - **Exception-handling part** (optional): handles exceptions that may arise during execution.

3. Parameter Modes:

- **IN**: Passes values to the procedure (acts like a constant).
- **OUT**: Returns values to the caller.
- **IN OUT**: Both passes initial values and returns updated ones.

4. Parameter Initialization:

- := or DEFAULT can be used to initialize IN parameters.

5. Autonomous Transactions:

- A procedure can be marked as autonomous using the PRAGMA AUTONOMOUS_TRANSACTION, allowing it to perform independent transactions from the main transaction.

6. AUTHID Clause:

- Determines whether a procedure is executed with the privileges of its **owner** or the **current user** (invoker).

7. Calling Procedures:

- Procedures can be invoked as simple PL/SQL statements, passing arguments through positional or named notation.

When would you use SQL Stored Procedures?



You would use SQL stored procedures when you want to:

Enhance security: By allowing controlled access to database operations and reducing the risk of SQL injection attacks.

Modularize code: To break down complex SQL logic into manageable, reusable modules for improved maintainability.

Improve performance: By reducing the overhead of repeatedly sending SQL statements to the database.

Automate tasks: For automating routine or complex database operations and data processing tasks.

Implement business logic: To encapsulate business rules and processes directly in the database.

Key Points about PL/SQL Procedure & Syntax

Detailed Explanation of Syntax Components:

1.CREATE [OR REPLACE]:

- **CREATE:** Initiates the creation of the procedure.
- **OR REPLACE:** Optional; if a procedure with the same name exists, it will be replaced.

2.PROCEDURE procedure_name:

- The name of the procedure; must adhere to Oracle's naming rules.

3.(parameter_name1 [mode] datatype, ...):

- **parameter_name:** The name of the parameter.
- **mode:** Defines how the parameter is used:
 - **IN:** Input parameter (default).
 - **OUT:** Output parameter.
 - **IN OUT:** Both input and output.
- **datatype:** The data type of the parameter (e.g., NUMBER, VARCHAR2).

4.IS:

- Indicates the start of the procedure body.

5.Declarative Part (optional):

- Local declarations for variables, constants, cursors, exceptions, etc.

6.BEGIN:

- Marks the beginning of the executable part.

7.Executable Part:

- Contains the SQL statements and PL/SQL logic that execute when the procedure is called.

8.EXCEPTION (optional):

- A section to handle exceptions that occur during execution.

9.END [procedure_name]:

- Marks the end of the procedure definition. You can optionally specify the procedure name again here.

10./:

- Used to execute the procedure definition in Oracle SQL.

```
CREATE [OR REPLACE] PROCEDURE procedure_name
[ (parameter_name1 IN (Default) datatype,
  parameter_name2 OUT datatype,
  parameter_name3 IN OUT datatype) ]
IS
    -- Declarative part (optional): declare variables, constants, cursors,
    -- exceptions, etc.
    variable_name datatype; -- Example of a variable declaration
BEGIN
    -- Executable part: SQL statements, PL/SQL logic
    -- Statements to perform operations, update data, etc.

    -- Example of SQL execution:
    -- INSERT INTO table_name (column1, column2) VALUES (value1,
    -- value2);

EXCEPTION
    -- Optional: Handle exceptions (errors) that occur during execution
    WHEN exception_name THEN
        -- Error handling logic
END procedure_name;
/
```

Representation of Parameters



Calling	Procedure
a 	X (IN)
b 	Y (OUT)
c 	Z (IN OUT)

Stored Procedure Example: a, b, and c

This stored procedure will take:

- **a (X - IN)**: An input parameter, the procedure will read this value.
- **b (Y - OUT)**: An output parameter, the procedure will set this value based on some logic.
- **c (Z - IN OUT)**: A parameter that will be both passed in and updated within the procedure.

Example of a Procedure Using IN, OUT, and IN OUT Parameters

Example of Each Parameter Mode

```
CREATE OR REPLACE PROCEDURE parameter_modes_demo (  
  p_in   IN   NUMBER,  -- IN parameter: can be read but not modified  
  p_out  OUT  NUMBER,  -- OUT parameter: can be modified and returned  
  p_inout IN OUT NUMBER -- IN OUT parameter: can be read, modified, and returned  
)  
IS  
BEGIN  
  -- IN parameter is read-only, we can't assign any value to it.  
  DBMS_OUTPUT.PUT_LINE('IN parameter (p_in) value: ' || p_in);  
  
  -- Assign a value to the OUT parameter  
  p_out := p_in * 2;  
  DBMS_OUTPUT.PUT_LINE('OUT parameter (p_out) value after modification: ' || p_out);  
  
  -- Modify the IN OUT parameter  
  p_inout := p_inout + 10;  
  DBMS_OUTPUT.PUT_LINE('IN OUT parameter (p_inout) value after modification: ' ||  
p_inout);  
END;  
/
```

Calling the Procedure

```
SET SERVEROUTPUT ON;  
  
DECLARE  
  l_in   NUMBER := 5;  
  l_out  NUMBER;  
  l_inout NUMBER := 3;  
BEGIN  
  -- Call the procedure  
  parameter_modes_demo(l_in, l_out, l_inout);  
  
  -- Display the results  
  DBMS_OUTPUT.PUT_LINE('OUT parameter returned: ' || l_out);  
  DBMS_OUTPUT.PUT_LINE('IN OUT parameter returned: ' || l_inout);  
END;  
/
```

Explanation:

- **IN** parameter (p_in): Passed a value (5) and used in the procedure, but not modified.
- **OUT** parameter (p_out): Not initialized when passed, but a value is assigned in the procedure and returned (10).
- **IN OUT** parameter (p_inout): Passed a value (3), modified inside the procedure (+10), and returned (13).

IN | OUT Parameter

IN Parameter ▪ Purpose: Used to pass data into the procedure. ▪ Behavior: The procedure can read the value, but it cannot modify it. Any changes to this parameter inside the procedure will not be reflected outside the procedure. ▪ Default: If you don't explicitly specify the parameter type, it is considered IN by default.	<pre>-- Procedure with IN Parameters (Insert a new country) CREATE OR REPLACE PROCEDURE Insert_Country (p_Country_Name IN VARCHAR2, p_Country_Code IN VARCHAR2) AS BEGIN INSERT INTO Countries (Country_Name, Country_Code) VALUES (p_Country_Name, p_Country_Code); COMMIT; END; /</pre>	<pre>-- Execution Example: BEGIN Insert_Country('Ghana', 'Gh'); END; /</pre>
OUT Parameter ▪ Purpose: Used to return data from the procedure. ▪ Behavior: The procedure can assign a value to the OUT parameter, and that value will be returned to the caller. The OUT parameter does not hold any value initially; it is populated inside the procedure. ▪ In this example, salary is an OUT parameter. After calling the procedure, the caller can retrieve the salary of the employee based on the employee ID.	<pre>-- Procedure with OUT Parameters (Fetch department details by ID) CREATE OR REPLACE PROCEDURE Get_Department_Details (p_Department_ID IN NUMBER, p_Department_Name OUT VARCHAR2, p_Location OUT VARCHAR2) AS BEGIN SELECT Department_Name, Location INTO p_Department_Name, p_Location FROM Departments WHERE Department_ID = p_Department_ID; END; /</pre>	<pre>-- Execution Example: DECLARE v_Department_Name VARCHAR2(100); v_Location VARCHAR2(100); BEGIN Get_Department_Details(1, v_Department_Name, v_Location); DBMS_OUTPUT.PUT_LINE('Department Name: ' v_Department_Name); DBMS_OUTPUT.PUT_LINE('Location: ' v_Location); END; /</pre>

IN OUT Parameter

INOUT Parameter

- **Purpose:** Used to pass data into the procedure and return modified data back to the caller.
- **Behavior:** The **INOUT** parameter allows the procedure to take an initial value (passed by the caller), modify it, and return the modified value.
- **In this example,** bonus is an **INOUT** parameter. You pass a value (the bonus) into the procedure, the procedure updates the employee's bonus, and it returns the updated bonus value back to the caller.

--Procedure with IN OUT Parameters (Update employee salary and return new salary)

```
CREATE OR REPLACE PROCEDURE Update_Employee_Salary (  
    p_Employee_ID IN NUMBER,  
    p_New_Salary IN OUT NUMBER  
) AS  
BEGIN  
    UPDATE Employees  
    SET Salary = p_New_Salary  
    WHERE Employee_ID = p_Employee_ID;  
  
    COMMIT;  
  
    -- Return the updated salary  
    SELECT Salary  
    INTO p_New_Salary  
    FROM Employees  
    WHERE Employee_ID = p_Employee_ID;  
END;  
/
```

-- Execution Example:

```
DECLARE  
    v_New_Salary NUMBER := 90000;  
BEGIN  
    Update_Employee_Salary(1,  
    v_New_Salary);  
    DBMS_OUTPUT.PUT_LINE('Updated  
Salary: ' || v_New_Salary);  
END;  
/
```

Procedure to Insert Multiple Employees (con't)

```
-- First, define collection types to hold multiple employee records
CREATE OR REPLACE TYPE emp_id_t IS TABLE OF NUMBER;
CREATE OR REPLACE TYPE emp_fname_t IS TABLE OF VARCHAR2(100);
CREATE OR REPLACE TYPE emp_lname_t IS TABLE OF VARCHAR2(100);
CREATE OR REPLACE TYPE emp_email_t IS TABLE OF VARCHAR2(100);
CREATE OR REPLACE TYPE emp_salary_t IS TABLE OF NUMBER;

-- Now create the procedure
CREATE OR REPLACE PROCEDURE manage_employee_bulk (
    p_employee_ids IN emp_id_t,      -- Array of employee IDs
    p_first_names IN emp_fname_t,    -- Array of first names
    p_last_names IN emp_lname_t,     -- Array of last names
    p_emails IN emp_email_t,         -- Array of emails
    p_salaries IN emp_salary_t       -- Array of salaries
)
IS
BEGIN
    -- Loop through the collection and perform insert for each record
    FOR i IN 1 .. p_employee_ids.COUNT LOOP
        INSERT INTO employees (employee_id, first_name, last_name, email, salary)
        VALUES (p_employee_ids(i), p_first_names(i), p_last_names(i), p_emails(i), p_salaries(i));

        DBMS_OUTPUT.PUT_LINE('Employee ' || p_employee_ids(i) || ' inserted successfully.');
```

```
    COMMIT;

EXCEPTION
    WHEN OTHERS THEN
        -- Handle exceptions and rollback in case of an error
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error occurred: ' || SQLERRM);
END manage_employee_bulk;
/
```

Explanation:

- **Collection Types:** We define collection types *emp_id_t*, *emp_fname_t*, *emp_lname_t*, *emp_email_t*, and *emp_salary_t* for the employee fields. Each type can hold an array of values.
- **Procedure Logic:** Inside the procedure, we loop through each collection using a FOR loop and insert one employee record at a time.
- **COMMIT/ROLLBACK:** After all inserts, the procedure commits the transaction. In case of any error, the transaction is rolled back.

Running the Procedure for Multiple Inserts

You can now call this procedure and pass multiple employee records using collections. Here's how to do it:

```
DECLARE
  -- Declare the collections to hold the employee data
  emp_ids emp_id_t := emp_id_t(1001, 1002, 1003);
  first_names emp_fname_t := emp_fname_t('John', 'Jane', 'Mike');
  last_names emp_lname_t := emp_lname_t('Doe', 'Smith', 'Johnson');
  emails emp_email_t := emp_email_t('john.doe@example.com', 'jane.smith@example.com', 'mike.johnson@example.com');
  salaries emp_salary_t := emp_salary_t(50000, 60000, 70000);
BEGIN
  -- Call the procedure with the collections
  manage_employee_bulk(emp_ids, first_names, last_names, emails, salaries);
END;
/
```

Explanation of the Call:

- **Collections:** You declare collections for each field and populate them with employee data (3 employees in this case).
- **Procedure Call:** The procedure is called with the collections, and it inserts all the employees in a single execution.

Creating a View with a Procedure

```
CREATE OR REPLACE PROCEDURE create_employee_view
IS
BEGIN
    -- Use dynamic SQL to create the view
    EXECUTE IMMEDIATE '
        CREATE OR REPLACE VIEW employee_view AS
        SELECT
            SALARY,
            BONUS,
            EMPLOYEE_ID,
            FIRST_NAME,
            LAST_NAME,
            EMAIL,
            PHONE_NUMBER,
            ADDRESS,
            HIRE_DATE,
            DEPARTMENT_ID,
            ROLE_ID
        FROM EMPLOYEES';

    -- Optional: Print confirmation
    DBMS_OUTPUT.PUT_LINE('View "employee_view" created successfully.');
```

■ Execute the Procedure

```
SET SERVEROUTPUT ON; -- To see the DBMS_OUTPUT message
```

```
BEGIN
    create_employee_view;
END;
/
```

-- View the Data: After executing the procedure, run this query to display the contents of the employee_view.

```
SELECT * FROM employee_view;
```

Note:

- **Updatable Views:** If a view is simple (based on a single table, no complex joins, aggregations, or functions), updating, inserting, or deleting data in the view will directly modify the data in the underlying table.
- **Non-Updatable Views:** If a view contains complex elements like joins, groupings, aggregations, or derived columns, it typically cannot be updated. Attempting to update such views will result in an error.

A **view** in SQL is a virtual table that is based on the result of a SELECT query. It does not store the data itself but rather provides a way to simplify complex queries by presenting data from one or more tables in a structured format. Essentially, a view is a saved query that can be treated like a table when retrieving or modifying data (depending on whether it's updatable or not).

The advantages of using views



Simplifies Complex Queries: Views can encapsulate complex SELECT statements, making it easier to work with large or complicated queries.

Data Security: Views can restrict access to specific columns or rows, providing a way to limit sensitive data exposure.

Data Abstraction: Views allow users to interact with data at a higher level, without needing to know the underlying table structure or relationships.

Reusability: Once created, a view can be reused in multiple queries, reducing duplication of complex logic.

Maintenance: Views make it easier to modify the underlying schema without affecting user queries, as the view can mask changes in the table structure.

Performance Optimization: In some cases, views can help optimize query performance, especially when using materialized views, which store the result set physically.

Create PL/SQL Function (1/2)



In PL/SQL, **functions** are similar to procedures, but they return a single value to the caller. Functions can take input parameters, perform calculations or operations, and then return a result. Functions are often used when a specific value needs to be calculated or returned, such as summing up values, performing a lookup, or processing data.

Key Points about PL/SQL Functions:

- Functions must return a value using the RETURN keyword.
- Functions can be used in SQL queries (SELECT, WHERE, etc.) and procedural PL/SQL blocks.
- A function can have input parameters but not output parameters.
- Functions must return a value of a specified data type.

Syntax for Creating a PL/SQL Function

While creating a function, you give a definition of what the function has to do.

```
CREATE OR REPLACE FUNCTION function_name (  
    parameter_name datatype -- Input parameter for the function  
)  
RETURN return_datatype -- Data type of the value the function will return  
AS  
    -- Local variable declarations (optional)  
    -- local_variable datatype;  
BEGIN  
    -- Function logic  
    -- Example: Perform some calculation or operation  
    -- local_variable := some_operation(parameter_name);  
  
    -- Compute and return the result  
    RETURN result_value; -- Return the final computed result  
  
    -- Comments explaining the function logic can be added here for clarification  
END;  
/
```

Create PL/SQL Function (2/2)

```
DECLARE
  a NUMBER; -- Variable to hold the first number
  b NUMBER; -- Variable to hold the second number
  c NUMBER; -- Variable to hold the maximum value

-- Function to find the maximum of two numbers
FUNCTION findMax(x IN NUMBER, y IN NUMBER)
RETURN NUMBER
IS
  z NUMBER; -- Local variable to hold the maximum value
BEGIN
  -- Compare the two numbers and assign the greater one to z
  IF x > y THEN
    z := x; -- Assign x to z if x is greater
  ELSE
    z := y; -- Assign y to z if y is greater
  END IF;

  RETURN z; -- Return the maximum value
END;

--- Calling the function
BEGIN
  a := 23; -- Assign value to a
  b := 45; -- Assign value to b

  -- Call the findMax function and store the result in c
  c := findMax(a, b);

  -- Output the maximum value
  DBMS_OUTPUT.PUT_LINE('Maximum of (23, 45): ' || c);
END;
```

- **function-name** specifies the name of the function.
- **[OR REPLACE]** option allows the modification of an existing function.
- The optional parameter list contains name, mode and types of the parameters. IN represents the value that will be passed from outside and OUT represents the parameter that will be used to return a value outside of the procedure.
- The function must contain a **return** statement.
- The **RETURN** clause specifies the data type you are going to return from the function.
- **function-body** contains the executable part.
- The AS keyword is used instead of the IS keyword for creating a standalone function.



Declaring, Defining, and Invoking a Simple PL/SQL Function that computes and returns the maximum of two values.

Oracle Database SQL Functions Overview

SQL In-Built Functions in Oracle

Functions are crucial for data processing and reporting in organizations. Oracle provides a variety of predefined functions to manipulate character, numeric, and date data. These functions help format and present data in the desired forms.



Types of In-Built Functions in Oracle

The in-built functions in Oracle are categorized into two main types:

- **Single Row Functions**
- **Multiple Row or Group Functions (Aggregate Functions)**

1. Single Row Functions

Single Row Functions operate on individual rows of data from the table based on filter criteria. Each function has a defined set of arguments, typically the column names, on which it performs operations.

Subcategories of Single Row Functions:

- **Character Functions:**
 - Manipulate string data (e.g., concatenation, finding string length).
- **Numeric Functions:**
 - Perform mathematical or arithmetic operations on numeric values in table columns.
- **Date Functions:**
 - Handle date manipulations (e.g., calculating days between dates, extracting parts of a date).
- **Conversion Functions:**
 - Convert one data type to another (e.g., from string to number).

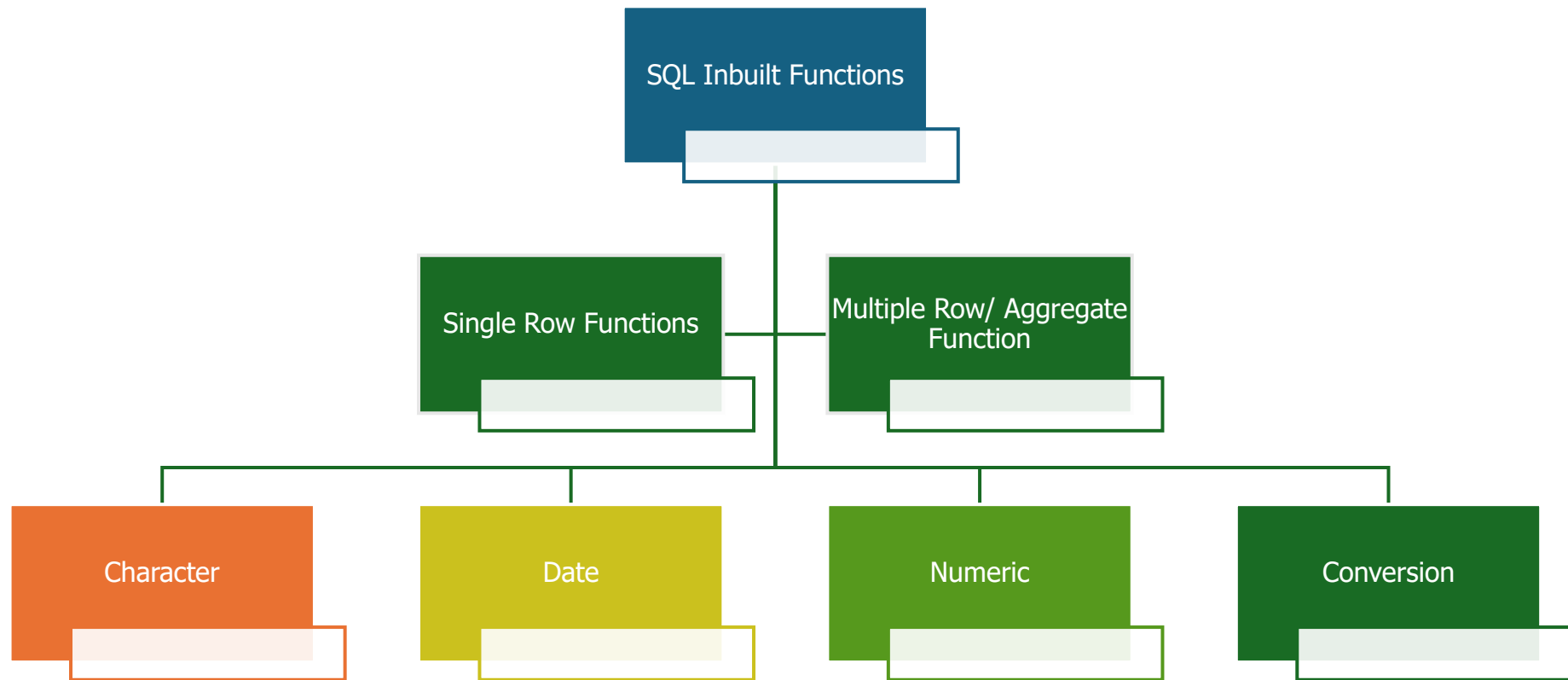
2. Multiple Row or Group Functions (Aggregate Functions)

These functions act on a set of rows returned by a SELECT statement, based on filter conditions applied in the WHERE clause. Aggregate functions take multiple values as arguments and return a single value as output, summarizing the results.

Examples of Aggregate Functions:

- **SUM:** Calculates the total of a specified column (e.g., total salaries).
- **AVG:** Computes the average value of a specified column.
- **COUNT:** Counts the number of rows that meet the specified criteria.

Oracle Database SQL Functions Overview



Creating and Using Functions in Oracle SQL Database

Category	Description	Examples of Functions
Single-Row Functions	Functions that return a single result for each row processed.	
Numeric Functions	Perform calculations on numeric data.	- ABS: Returns the absolute value. - ROUND: Rounds a number to a specified decimal place. - SQRT: Returns the square root.
Character Functions	Operate on strings and return string values.	- UPPER: Converts a string to uppercase. - SUBSTR: Returns a substring from a string. - CONCAT: Combines two strings.
Datetime Functions	Work with date and time data types.	- CURRENT_DATE: Returns the current date and time. - ADD_MONTHS: Adds a specified number of months to a date. - LAST_DAY: Returns the last day of the month for a given date.
Conversion Functions	Convert values from one datatype to another.	- TO_CHAR: Converts a number or date to a string. - TO_NUMBER: Converts a string to a number. - CAST: Converts an expression from one datatype to another.
Miscellaneous Functions	Various functions that do not fit into the above categories.	- NVL: Returns a specified value if the expression is null. - COALESCE: Returns the first non-null expression. - DECODE: Performs conditional logic based on input.
Aggregate Functions	Return a single result based on a group of rows, typically used with GROUP BY.	- SUM: Returns the sum of values in a group. - AVG: Calculates the average of values in a group. - COUNT: Counts the number of rows in a group.
Analytic Functions	Similar to aggregate functions, but return multiple rows for each group.	- RANK: Assigns a rank to each row within a partition. - LEAD: Accesses data from the next row in the same result set. - LAG: Accesses data from the previous row in the same result set.

Functions in Oracle SQL Database – Examples (1/4)

■ Numeric Functions

ABS: Returns the absolute value of a number.



 **Syntax:** ABS(number)

 **Example:** SELECT ABS(-5) FROM dual;
— **Output: 5**

ROUND: Rounds a number to a specified number of decimal places.



 **Syntax:** ROUND(number, decimals)

 **Example:** SELECT ROUND(45.678, 2) FROM dual;
— **Output: 45.68**

SQRT: Returns the square root of a number.



 **Syntax:** SQRT(number)

 **Example:** SELECT SQRT(16) FROM dual;
— **Output: 4**



■ Character Functions

UPPER: Converts a string to uppercase.



 **Syntax:** UPPER(string)

 **Example:** SELECT UPPER('hello') FROM dual;
— **Output: HELLO**

SUBSTR: Returns a substring from a string.



 **Syntax:** SUBSTR(string, start, length)

 **Example:** SELECT SUBSTR('Oracle', 2, 3) FROM dual;
— **Output: rac**

CONCAT: Combines two strings into one.



 **Syntax:** CONCAT(string1, string2)

 **Example:** SELECT CONCAT('Oracle', 'PL/SQL') FROM dual;
— **Output: OraclePL/SQL**

Functions in Oracle SQL Database – Examples (2/4)



Datetime Functions

CURRENT_DATE: Returns the current date and time of the session.



Syntax: CURRENT_DATE

Example: SELECT CURRENT_DATE FROM dual; — **Output:** Current date and time

ADD_MONTHS: Adds a specific number of months to a date.



Syntax: ADD_MONTHS(date, number_of_months)

Example: SELECT ADD_MONTHS(SYSDATE, 3) FROM dual; — **Output:** Date after 3 months.

LAST_DAY: Returns the last day of the month for a given date.



Syntax: LAST_DAY(date)

Example: SELECT LAST_DAY(SYSDATE) FROM dual; — **Output:** Last day of the current month.

Conversion Functions

TO_CHAR: Converts a number or date to a string.



Syntax: TO_CHAR(expression, format)

Example: SELECT TO_CHAR(SYSDATE, 'DD-MON-YYYY') FROM dual; — **Output:** 11-OCT-2024

TO_NUMBER: Converts a string to a number.



Syntax: TO_NUMBER(string)

Example: SELECT TO_NUMBER('12345') FROM dual; — **Output:** 12345

CAST: Converts an expression from one datatype to another.



Syntax: CAST(expression AS datatype)

Example: SELECT CAST('12345' AS NUMBER) FROM dual; — **Output:** 12345

Functions in Oracle SQL Database – Examples (3/4)

Miscellaneous Functions

NVL: Replaces NULL with a specified value.



Syntax: NVL(expr1, expr2)

Example: SELECT NVL(NULL, 'default') FROM dual; — **Output: default**

COALESCE: Returns the first non-NULL expression.



Syntax: COALESCE(expr1, expr2, ..., exprN)

Example: SELECT COALESCE(NULL, NULL, 'value') FROM dual; — **Output: value**

DECODE: Performs conditional logic.



Syntax: DECODE(expr, search, result, default)

Example: SELECT DECODE(1, 1, 'One', 'Other') FROM dual; — **Output: One**



Aggregate Functions (Used with GROUP BY)

SUM: Returns the sum of values in a group.



Syntax: SUM(expression)

Example: SELECT SUM(salary) FROM employees; — **Output: Sum of salaries.**

AVG: Calculates the average of values in a group.



Syntax: AVG(expression)

Example: SELECT AVG(salary) FROM employees; — **Output: Average salary.**

COUNT: Counts the number of rows in a group.



Syntax: COUNT(expression)

Example: SELECT COUNT(*) FROM employees; — **Output: Number of employees.**

Functions in Oracle SQL Database – Examples (4/4)

▪ Analytic Functions



RANK: Assigns a rank to each row within a partition.



 **Syntax:** RANK() OVER (PARTITION BY expr ORDER BY expr)

 **Example:** SELECT RANK() OVER (ORDER BY salary DESC) FROM employees; — **Ranks employees based on salary.**

LEAD: Accesses data from the next row in the result set.



 **Syntax:** LEAD(expr, offset) OVER (ORDER BY expr)

 **Example:** SELECT LEAD(salary, 1) OVER (ORDER BY salary) FROM employees; — **Retrieves the next salary in the sorted list.**

LAG: Accesses data from the previous row in the result set.



 **Syntax:** LAG(expr, offset) OVER (ORDER BY expr)

 **Example:** SELECT LAG(salary, 1) OVER (ORDER BY salary) FROM employees; — **Retrieves the previous salary in the sorted list.**

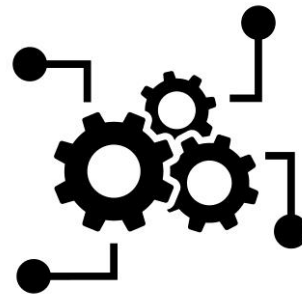
PL/SQL Function - Example

Function to Get Employee Full Name

```
CREATE OR REPLACE FUNCTION Get_Full_Name (  
    p_First_Name IN VARCHAR2,  
    p_Last_Name IN VARCHAR2  
) RETURN VARCHAR2  
AS  
    v_Full_Name VARCHAR2(100);  
BEGIN  
    -- Concatenate the first name and last name  
    v_Full_Name := p_First_Name || ' ' || p_Last_Name;  
  
    -- Return the full name  
    RETURN v_Full_Name;  
END;  
/
```

Calling Get_Full_Name

```
DECLARE  
    v_Full_Name VARCHAR2(100);  
BEGIN  
    -- Call the function and store the result in a variable  
    v_Full_Name := Get_Full_Name('Jane', 'Smith');  
  
    -- Output the result  
    DBMS_OUTPUT.PUT_LINE('Full Name: ' || v_Full_Name);  
END;  
/
```



PL/SQL Function - Example

This function takes a Role_ID as input and returns the total allowance amount for that role.

```
CREATE OR REPLACE FUNCTION Get_Total-Allowance (  
    p_Role_ID NUMBER -- Input parameter for Role_ID  
)  
RETURN NUMBER -- The data type of the value the function will return  
AS  
    v_Total-Allowance NUMBER := 0; -- Variable to hold the computed total allowance  
BEGIN  
    -- Calculate the total allowance amount for the given Role_ID  
    SELECT SUM(Allowance_Amount) INTO v_Total-Allowance  
    FROM Allowances  
    WHERE Role_ID = p_Role_ID; -- Filter by the input Role_ID  
  
    -- If no allowance records exist for the Role_ID, return 0  
    RETURN NVL(v_Total-Allowance, 0); -- Return the total allowance, defaulting to 0 if  
null  
EXCEPTION  
    WHEN OTHERS THEN  
        -- Handle any exceptions that might occur  
        RETURN -1; -- Return -1 to indicate an error  
END Get_Total-Allowance;  
/
```

Call this function as follows:

```
DECLARE  
    v_Total NUMBER; -- Variable to hold the result  
BEGIN  
    -- Call the function with a specific Role_ID  
    v_Total := Get_Total-Allowance(1); -- Replace '1' with the desired Role_ID  
    DBMS_OUTPUT.PUT_LINE('Total Allowance: ' || v_Total);  
END;  
/
```

PL/SQL Using Aggregate Functions (SUM, AVG) in a Procedure

This function takes a total Salary and Avg Salary as input (Multiple row Function)

```
CREATE OR REPLACE PROCEDURE get_salary_stats (  
    total_sal OUT NUMBER,  
    avg_sal OUT NUMBER  
) AS  
BEGIN  
    -- Use SELECT INTO to assign aggregate results to variables  
    SELECT SUM(salary), AVG(salary)  
    INTO total_sal, avg_sal  
    FROM employees;  
  
    -- Optionally, display the results  
    DBMS_OUTPUT.PUT_LINE('Total Salary: ' || total_sal);  
    DBMS_OUTPUT.PUT_LINE('Average Salary: ' || avg_sal);  
END;  
/
```

Call this function as follows:

```
DECLARE  
    v_total_salary NUMBER;  
    v_avg_salary NUMBER;  
BEGIN  
    -- Call the procedure and pass the variables to hold the OUT parameters  
    get_salary_stats(v_total_salary, v_avg_salary);  
  
    -- Optionally display the values in the anonymous block  
    DBMS_OUTPUT.PUT_LINE('Total Salary: ' || v_total_salary);  
    DBMS_OUTPUT.PUT_LINE('Average Salary: ' || v_avg_salary);  
END;  
/
```

PL/SQL Recursive Functions

--This function computes the factorial of a given number using recursion.

```
CREATE OR REPLACE FUNCTION factorial_func (x NUMBER)
RETURN NUMBER
IS
    f NUMBER; -- Local variable to hold the factorial result
BEGIN
    -- Base case: if x is 0, return 1 (because 0! = 1)
    IF x = 0 THEN
        f := 1;
    ELSE
        -- Recursive case: x * factorial of (x-1)
        f := x * factorial_func(x - 1);
    END IF;

    -- Return the result
    RETURN f;
END factorial_func;
/

--- calling the factorial function

DECLARE
    num NUMBER := 5;      -- Input number for factorial calculation
    factorial_result NUMBER; -- Variable to store the result
BEGIN
    -- Call the function and assign the result to factorial_result
    factorial_result := factorial_func(num);

    -- Output the result using DBMS_OUTPUT
    DBMS_OUTPUT.PUT_LINE('Factorial of ' || num || ' is: ' || factorial_result);
END;
/
```

A **recursive function** is a function that calls itself in order to solve a problem. It breaks the problem down into smaller, more manageable sub-problems, solving each one until a base case is reached. Recursive functions are commonly used in programming for tasks that can naturally be divided into similar subtasks, such as calculating factorials, traversing data structures (like trees and graphs), and solving problems that involve backtracking.

How It Works:

- **Base Case:** If the input n is 0, the function returns 1, as $0! = 1$.
- **Recursive Case:** If $n > 0$, the function multiplies n by the factorial of $n-1$, i.e., $f = n * \text{factorial_func}(n-1)$.

This process continues until the base case ($n = 0$) is reached, at which point the function stops recursing and returns the final result.

The following program calculates the factorial of a given number by calling itself recursively –

PL/SQL Recursive Functions - Fibonacci



```
CREATE OR REPLACE FUNCTION fibonacci_func(n NUMBER)
RETURN NUMBER
IS
    f NUMBER; -- Local variable to hold the Fibonacci result
BEGIN
    -- Base case: F(0) = 0 and F(1) = 1
    IF n = 0 THEN
        f := 0;
    ELSIF n = 1 THEN
        f := 1;
    ELSE
        -- Recursive case: F(n) = F(n-1) + F(n-2)
        f := fibonacci_func(n - 1) + fibonacci_func(n - 2);
    END IF;

    -- Return the computed Fibonacci value
    RETURN f;
END fibonacci_func;
/

-- Calling the function
DECLARE
    num NUMBER := 7;      -- Input number for Fibonacci calculation
    fibonacci_result NUMBER; -- Variable to store the result
BEGIN
    -- Call the function and assign the result to fibonacci_result
    fibonacci_result := fibonacci_func(num);

    -- Output the result using DBMS_OUTPUT
    DBMS_OUTPUT.PUT_LINE('Fibonacci of ' || num || ' is: ' || fibonacci_result);
END;
/

-- Option 2: Call in a SQL Query
SELECT fibonacci_func(7) AS fibonacci_result FROM dual;
```

For calculating the **Fibonacci sequence**. The Fibonacci sequence is defined as:

- $F(0) = 0$
- $F(1) = 1$
- $F(n) = F(n-1) + F(n-2)$ for $n > 1$

The following program calculates the fibonacci of a given number by calling itself recursively –

PL/SQL Procedure vs Function

Feature	Procedure	Function
Returns a Value	No (uses OUT parameters)	Yes (via return type)
Invoked in SQL Statements	No	Yes
Purpose	Perform an action or operation	Perform calculations and return a value
Parameter Modes	IN, OUT, IN OUT	Typically IN
Multiple Values	Can return multiple values via OUT params	Returns a single value
Side Effects	Can have side effects (e.g., modify data)	Should avoid side effects
Best Used For	Tasks that perform operations or change data	Tasks that compute and return data
DML Operations	Can perform DML and COMMIT/ROLLBACK	Can perform DML (restricted in SQL)
Calling Syntax	Called with EXEC or PL/SQL block	Called in SQL or PL/SQL block

Procedure vs Functions



Functions:

These subprograms return a single value; mainly used to compute and return a value.

Procedures:

These subprograms do not return a value directly; mainly used to perform an action.

Procedure:

Cannot be called from within a SQL statement (e.g., SELECT, INSERT, UPDATE, DELETE).

PL/SQL Procedure vs Function

```
-- Procedure to update the salary of an employee based on their employee ID
CREATE OR REPLACE PROCEDURE update_salary (
    p_emp_id    IN employees.employee_id%TYPE, -- Input parameter: employee ID
    p_new_salary IN employees.salary%TYPE      -- Input parameter: new salary
) IS
BEGIN
    -- Update the salary for the given employee ID
    UPDATE employees
    SET salary = p_new_salary
    WHERE employee_id = p_emp_id;
END;
/

-- Call the procedure to update salary
BEGIN
    update_salary(101, 5000); -- Update employee 101's salary to 5000
END;
/
```

```
-- Function to get the full name of an employee based on their employee ID
CREATE OR REPLACE FUNCTION get_full_name (
    p_emp_id IN employees.employee_id%TYPE -- Input parameter: employee ID
) RETURN VARCHAR2
IS
    v_full_name employees.first_name%TYPE || ' ' || employees.last_name%TYPE; --
    Declare variable to store full name
BEGIN
    -- Fetch the full name from the employees table
    SELECT first_name || ' ' || last_name
    INTO v_full_name
    FROM employees
    WHERE employee_id = p_emp_id;

    RETURN v_full_name; -- Return the full name
END;
/

-- Call the function to get the full name
BEGIN
    DBMS_OUTPUT.PUT_LINE(get_full_name(101)); -- Print the full name of employee
101
END;
/
```

PL/SQL Procedure – Triangle Patterns

This procedure will accept the number of rows for the triangle and print the pattern.

```
CREATE OR REPLACE PROCEDURE draw_triangle_pattern (p_rows IN NUMBER)
IS
BEGIN
    -- Loop through each row
    FOR i IN 1..p_rows LOOP
        -- Inner loop to print '*' characters
        FOR j IN 1..i LOOP
            DBMS_OUTPUT.PUT('*');
        END LOOP;

        -- Move to the next line after each row
        DBMS_OUTPUT.PUT_LINE("");
    END LOOP;
END draw_triangle_pattern;
/

-- Call the procedure
BEGIN
    draw_triangle_pattern(5); -- Call the procedure with 5 rows
END;
/
```

Explanation:

- **Outer Loop (FOR i IN 1..p_rows LOOP):** This loop controls the number of rows in the triangle.
For example, if p_rows = 5, the loop will run 5 times, once for each row.
- **Inner Loop (FOR j IN 1..i LOOP):** This inner loop prints the asterisks (*) for each row. The number of asterisks printed in each row increases with each iteration of the outer loop.
- **DBMS_OUTPUT.PUT and DBMS_OUTPUT.PUT_LINE:**
 - DBMS_OUTPUT.PUT prints characters on the same line.
 - DBMS_OUTPUT.PUT_LINE moves to the next line after printing a row.

Expected Output:

```
*
**
***
****
*****
```

PL/SQL Package

What is a PL/SQL Package?

In PL/SQL, a package is a schema object **that groups related functionalities**, including **variables, constants, cursors, exceptions, procedures, functions, and subprograms**. It is compiled and stored in the Oracle Database. A package typically consists of two parts:



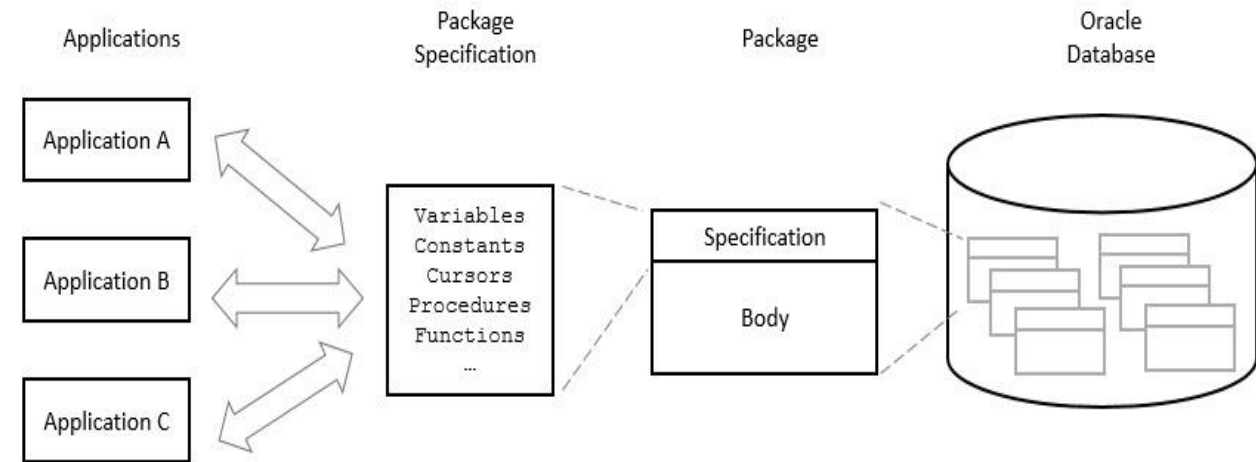
1. **Package Specification** (mandatory)
2. **Package Body** (optional, depending on the specification)

Package Specification

- The package specification declares public objects (variables, cursors, subprograms, etc.) that are accessible outside the package.
- If the public objects include cursors or subprograms, the package must have a body that defines them.

Package Body

- The package body contains the implementation of cursors or subprograms declared in the package specification.
- It may define private objects (variables, cursors) that are only accessible within the package body.
- The package body can also have:
 - **Initialization Part:** Executes statements that initialize variables or perform a one-time setup.
 - **Exception-Handling Part:** Manages exceptions within the package.



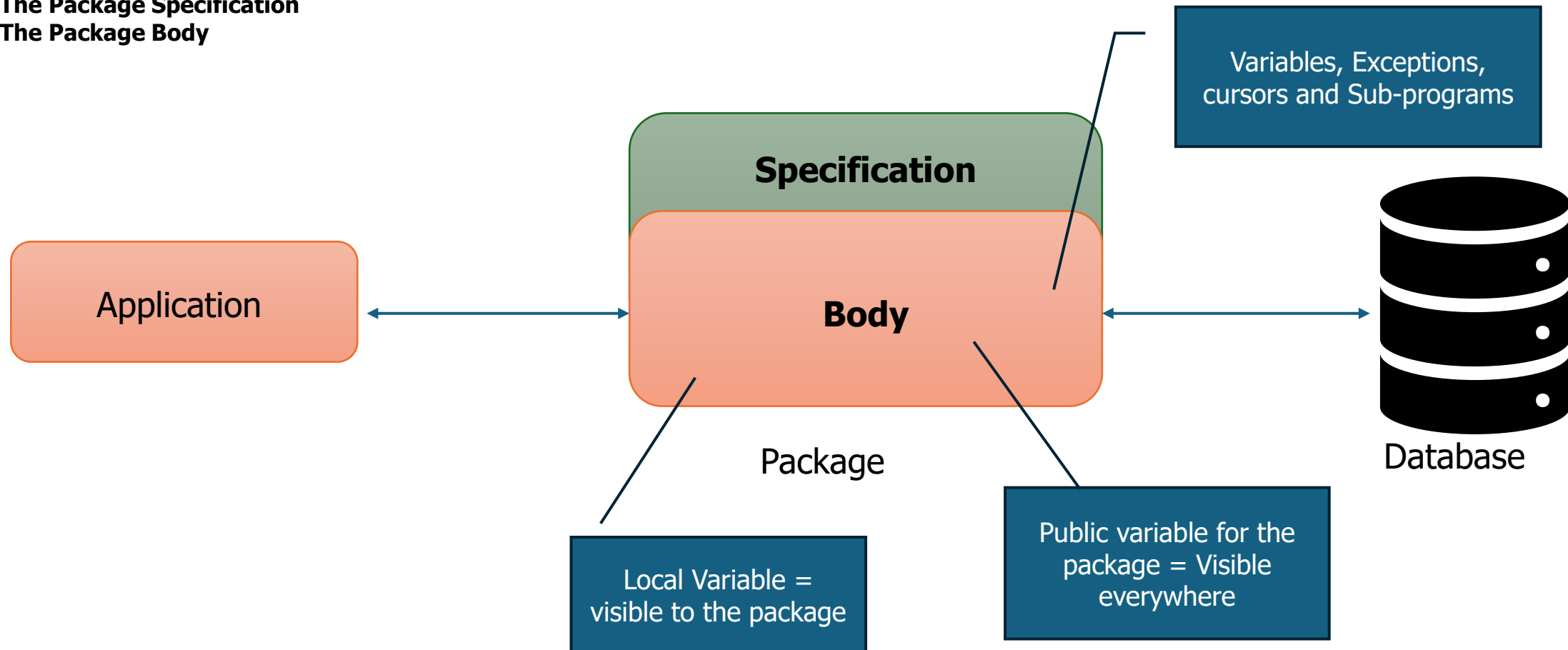
Source Image: <https://www.oracletutorial.com/plsql-tutorial/plsql-package/>

PL/SQL Package



A package in Oracle PL/SQL consists of two main parts:

1. **The Package Specification**
2. **The Package Body**



Advantages of PL/SQL Packages

Modular Code:

Packages allow you to group logically related objects, making the code reusable, manageable, and reliable.



Encapsulation and Hiding Implementation:

Packages expose functionality through the specification while hiding implementation details in the body.

This allows modifications to the package body without affecting other dependent packages or applications.

Improved Performance:

Oracle loads the entire package into memory upon the first call to a subprogram, reducing disk I/O for subsequent calls and improving performance.

Avoid Unnecessary Recompiling:

If you modify the body of a package, Oracle only recompiles that part, not the subprograms dependent on the package specification. This minimizes recompiling overhead.

Easy Authorization Management:

Packages allow you to grant permissions on the entire package rather than granting permissions on individual objects, simplifying security management.

Conclusion: PL/SQL packages encapsulate code and offer significant benefits like modularity, encapsulation, performance improvement, and easier management. These features make packages essential for building scalable, maintainable PL/SQL applications.

Syntax Create a package in PL/SQL – Package Specification



```
CREATE OR REPLACE PACKAGE package_name IS
  -- Declarations of public procedures, functions, constants, variables, types, etc.

  -- Example: Declare a constant
  constant_name CONSTANT datatype := value;

  -- Example: Declare a procedure
  PROCEDURE procedure_name (parameter1 datatype, parameter2 datatype);

  -- Example: Declare a function
  FUNCTION function_name (parameter1 datatype) RETURN datatype;

  -- Example: Declare a variable
  variable_name datatype;

  -- Example: Declare a type (optional)
  TYPE type_name IS RECORD (...);

  -- Other public elements can be declared here

END package_name;
```

Labeling:

- `CREATE OR REPLACE PACKAGE package_name IS:` This is the start of **the Package Specification**. The `IS` keyword introduces the declarations.
- **Declarations:** Here, you declare constants, variables, procedures, functions, types, etc., that will be accessible publicly.
- `END package_name;;` Ends the package specification.

A **TYPE** is generally used to define new types of data or to organize related data elements.

Step 1. Package Specification

The **Package Specification** defines the interface to the package. It declares all public procedures, functions, constants, types, and other elements that can be accessed by users or other PL/SQL programs.

Syntax Create a package in PL/SQL – Package Body



```
CREATE OR REPLACE PACKAGE BODY package_name IS
  -- Implementations of the procedures and functions

  -- Example: Implementation of the procedure
  PROCEDURE procedure_name (parameter1 datatype, parameter2 datatype) IS
  BEGIN
    -- Procedure logic
  END procedure_name;

  -- Example: Implementation of the function
  FUNCTION function_name (parameter1 datatype) RETURN datatype IS
  BEGIN
    -- Function logic
    RETURN value;
  END function_name;

  -- Example: Private procedures, variables (optional)
  PROCEDURE private_procedure IS
  BEGIN
    -- Logic for private procedure (only visible inside the package)
  END private_procedure;

  -- Private variables, types, and other elements (not visible outside the package)

END package_name;
```

Labeling:

- **CREATE OR REPLACE PACKAGE BODY package_name IS:** This is the start of the **Package Body**.
- **Implementations:** This section contains the full code for the procedures and functions declared in the specification. It can also contain private procedures and variables.
- **END package_name;** Ends the package body.

Step 2. Package Body

The **Package Body** contains the implementation of the procedures and functions declared in the specification. It can also contain private elements (procedures, variables, etc.) that are not exposed in the specification.

Syntax Create a package in PL/SQL – Package Body

Step 1. Package Specification

```
--First, drop the package if it exists
DROP PACKAGE employee_package;/

CREATE OR REPLACE PACKAGE employee_package IS
  -- Declare a constant
  max_bonus CONSTANT NUMBER := 5000;

  -- Declare a procedure to calculate bonus
  PROCEDURE calculate_bonus (p_salary IN NUMBER, p_bonus OUT
NUMBER);

  -- Declare a function to calculate tax
  FUNCTION calculate_tax (p_purchase_amount IN NUMBER)
RETURN NUMBER;

END employee_package;
/
```

Step 2. Package body

```
-- Next, create the package body
CREATE OR REPLACE PACKAGE BODY employee_package IS
  PROCEDURE calculate_bonus (p_salary IN NUMBER, p_bonus OUT NUMBER) IS
  BEGIN
    p_bonus := p_salary * 0.10; -- 10% bonus
    IF p_bonus > max_bonus THEN
      p_bonus := max_bonus; -- Cap the bonus at max_bonus
    END IF;
  END calculate_bonus;

  FUNCTION calculate_tax (p_purchase_amount IN NUMBER) RETURN NUMBER IS
    v_tax NUMBER;
  BEGIN
    v_tax := p_purchase_amount * 0.15; -- 15% tax
    RETURN v_tax;
  END calculate_tax;

  PROCEDURE log_error IS
  BEGIN
    NULL; -- Placeholder for private procedure logic
  END log_error;
END employee_package;
/
```

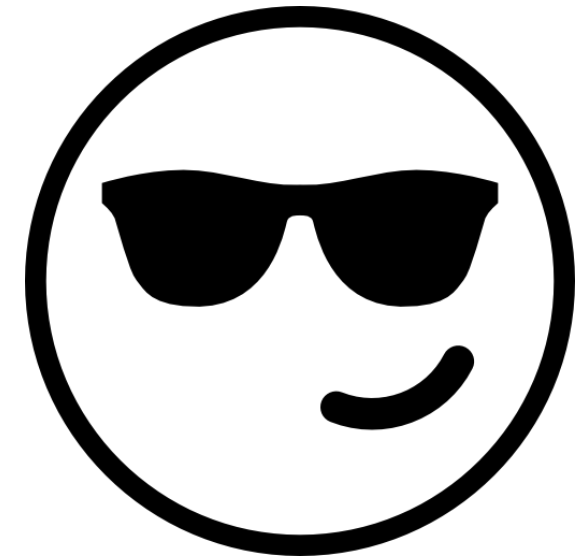


Syntax Create a package in PL/SQL – Package Body

Step 3. Calling the package

```
DECLARE
  v_bonus NUMBER;
  v_tax   NUMBER;
BEGIN
  -- Call the calculate_bonus procedure
  employee_package.calculate_bonus(5000, v_bonus);
  DBMS_OUTPUT.PUT_LINE('Calculated Bonus: ' || v_bonus);

  -- Call the calculate_tax function
  v_tax := employee_package.calculate_tax(1000);
  DBMS_OUTPUT.PUT_LINE('Calculated Tax: ' || v_tax);
END;
```



PL/SQL Package public (global) and local (private) variables

In PL/SQL, a **package** can contain both **public** (global) and **local** (private) variables. Public variables are accessible to any code that can access the package, whereas local variables are only accessible within the package body.



Creating the Package Specification

```
CREATE OR REPLACE PACKAGE employee_pkg AS
  -- Public variable (accessible outside the package)
  emp_count NUMBER := 0;

  -- Public procedure
  PROCEDURE add_employee;
END employee_pkg;
/
```

Creating the Package Body

```
CREATE OR REPLACE PACKAGE BODY employee_pkg AS
  -- Local variable (only accessible within the package body)
  emp_limit NUMBER := 100;

  -- Procedure implementation
  PROCEDURE add_employee IS
  BEGIN
    -- Increment the public variable
    emp_count := emp_count + 1;

    -- Check the local variable limit
    IF emp_count > emp_limit THEN
      DBMS_OUTPUT.PUT_LINE('Employee limit reached.');
```

PL/SQL Package public (global) and local (private) variables



Explanation

- **Public Variable (emp_count):** Declared in the package specification, it can be accessed and modified from outside the package (e.g., by calling employee_pkg.emp_count).
- **Local Variable (emp_limit):** Declared in the package body, it is only accessible within the package body. This variable cannot be accessed outside the package, and is useful for internal control.

Using the Package

```
BEGIN
  employee_pkg.add_employee; -- Adds an employee and updates emp_count
  DBMS_OUTPUT.PUT_LINE('Total Employees: ' || employee_pkg.emp_count); --
  Accesses the public variable
END;
/
```

PL/SQL Encapsulation – Example (1/4)

Step 1: Updated PL/SQL Package: Package Specification



```
/*
Step 1: Updated PL/SQL Package: Package Specification
*/

CREATE OR REPLACE PACKAGE emp_mgmt AS
  -- Procedure to add a new employee to the EMPLOYEES table
  PROCEDURE add_employee(
    p_first_name  VARCHAR2, -- First name of the employee
    p_last_name   VARCHAR2, -- Last name of the employee
    p_email       VARCHAR2, -- Email address of the employee
    p_phone_number VARCHAR2, -- Phone number of the employee
    p_address     VARCHAR2, -- Residential address of the employee
    p_hire_date   DATE,      -- Hire date of the employee
    p_department_id NUMBER,  -- Department ID the employee belongs to
    p_role_id     NUMBER,    -- Role ID of the employee
    p_salary      NUMBER     -- Salary of the employee
  );

  -- Function to retrieve the department name based on employee ID
  FUNCTION get_department(
    p_emp_id NUMBER -- Employee ID for which to retrieve the department name
  ) RETURN VARCHAR2; -- Returns the department name

  -- Additional procedures/functions can be added here as needed
END emp_mgmt;
/
```

PL/SQL Encapsulation (2/4)

In PL/SQL, using a package is called encapsulation because it groups related procedures, functions, and data together while hiding the implementation details. This ensures:

Data Hiding: Internal logic is concealed, exposing only necessary parts.



Modularity: Related functionality is bundled, promoting cleaner code.

Access Control: Only defined interfaces are accessible, preventing misuse.

Reusability: Encapsulated units can be reused without exposing internals.

Integrity: The package enforces rules and consistency, protecting data.

In essence, packages provide abstraction and control, making code more organized, secure, and maintainable.

PL/SQL Encapsulation – Example (3/4)

Step 2: Body Package



```
CREATE OR REPLACE PACKAGE BODY emp_mgmt AS
```

```
-- Implementation of the procedure to add a new employee
PROCEDURE add_employee(
    p_first_name  VARCHAR2, -- First name of the employee
    p_last_name   VARCHAR2, -- Last name of the employee
    p_email       VARCHAR2, -- Email address of the employee
    p_phone_number VARCHAR2, -- Phone number of the employee
    p_address     VARCHAR2, -- Residential address of the employee
    p_hire_date   DATE,      -- Hire date of the employee
    p_department_id NUMBER,  -- Department ID the employee
    belongs to
    p_role_id     NUMBER, -- Role ID of the employee
    p_salary      NUMBER  -- Salary of the employee
) IS
BEGIN
    -- Insert the new employee record into the EMPLOYEES table
    INSERT INTO employees (
        first_name, last_name, email, phone_number, address,
        hire_date,
        department_id, role_id, salary
    ) VALUES (
        p_first_name, p_last_name, p_email, p_phone_number,
        p_address, p_hire_date, p_department_id, p_role_id, p_salary
    );
END add_employee;
```

```
-- Implementation of the function to retrieve the department name
FUNCTION get_department(p_emp_id NUMBER) RETURN VARCHAR2 IS
    v_department_name VARCHAR2(100); -- Variable to hold the
    department name
BEGIN
    -- Retrieve the department name based on the employee's department
    ID
    SELECT department_name
    INTO v_department_name
    FROM departments
    WHERE department_id = (
        SELECT department_id
        FROM employees
        WHERE employee_id = p_emp_id
    );
    RETURN v_department_name; -- Return the retrieved department
    name
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        -- Handle case where no department is found for the given employee
        ID
        RETURN NULL;
    END get_department;

END emp_mgmt;
/
```

PL/SQL Encapsulation – Example (4/4)

Step 3 : Using the Package

```
/*
Using the Package
*/

BEGIN
  -- Adding a new employee named John Doe with specific details
  emp_mgmt.add_employee(
    p_first_name  => 'John',
    p_last_name   => 'Doe',
    p_email       => 'john.doe@example.com',
    p_phone_number => '123-456-7890',
    p_address     => '123 Elm Street, Cityville',
    p_hire_date   => TO_DATE('2023-06-15', 'YYYY-MM-DD'),
    p_department_id => 3, -- Assuming 3 is the department ID for IT
    p_role_id     => 5,   -- Assuming 5 is the role ID for Software Engineer
    p_salary      => 65000 -- Salary amount in the local currency
  );
END;
/
```

```
/*
Retrieving the Department Name for an Employee
To get the department name for a specific employee based on their employee ID,
you can call the get_department function. Below is an example of how to use the function.
*/

DECLARE
  v_dept_name VARCHAR2(100); -- Variable to hold the department name
BEGIN
  -- Get the department name for employee with ID 1001
  v_dept_name := emp_mgmt.get_department(p_emp_id => 3);

  -- Display the result
  DBMS_OUTPUT.PUT_LINE('Department Name: ' || v_dept_name);
END;
/
```

Step4: Testing the query

```
/*
Testing the Code
*/
```

```
SELECT * FROM EMPLOYEES WHERE DEPARTMENT_ID = 3;
```

Object Type

Example

```
CREATE TYPE employee_type AS OBJECT (  
    employee_id NUMBER,  
    first_name VARCHAR2(50),  
    last_name VARCHAR2(50),  
    salary NUMBER  
);
```

```
DECLARE  
    emp employee_type;  
BEGIN  
    emp := employee_type(101, 'John', 'Doe', 5000);  
END;
```

- An **Object Type** allows you to define a structure that encapsulates data and methods (procedures and functions) within it.
- You define object types using the CREATE TYPE statement, and the type can have attributes (fields) and methods (functions or procedures).



Object Types - Collection Types

Collection Types

Collections are ordered groups of elements, and Oracle supports three types of collections:

VARRAY (Variable-Size Arrays)

Nested Tables

Associative Arrays (formerly known as PL/SQL Tables or Index-by Tables)

VARRAY: A fixed-size, ordered collection. You specify the maximum number of elements it can hold.

Nested Table: A collection that can hold an arbitrary number of elements. It is more flexible than a VARRAY but has some performance overhead.

Associative Arrays: Similar to hash tables or dictionaries, with key-value pairs.

In PL/SQL, an object collection is a data structure that allows you to store collections of objects. It can be useful for managing complex data structures where each element in the collection is itself an object, encapsulating data and methods. PL/SQL offers two main types of collections: **nested tables** and **VARRAYs**. Here's how you can define and use an object collection in PL/SQL.

Object Types & Collection Example(1/2)

1. Define an Object Type

First, you define an object type that describes the structure of each element in the collection.

```
CREATE OR REPLACE TYPE employee_type AS OBJECT (  
    employee_id  NUMBER,  
    first_name   VARCHAR2(50),  
    last_name    VARCHAR2(50),  
    salary       NUMBER  
);
```

2. Define a Collection Type of the Object

After defining the object type, you create a collection type based on it. This collection type can be either a **nested table** or a **VARRAY**.

Using a Nested Table

A nested table is an unbounded collection (size can vary) of objects.

```
CREATE OR REPLACE TYPE employee_table AS TABLE OF employee_type;
```

Using a VARRAY

A VARRAY (Variable Array) is a bounded collection with a fixed size limit.

```
CREATE OR REPLACE TYPE employee_table AS TABLE OF employee_type;
```

3. Using the Object Collection in PL/SQL

You can use these types in PL/SQL blocks to manipulate collections of objects. Here's an example that demonstrates creating an *employee_table*, adding elements, and iterating through the collection.

```
DECLARE  
    employees employee_table := employee_table(); -- Initializing  
    the nested table  
BEGIN  
    -- Add elements to the collection  
    employees.EXTEND(2); -- Extend the table to hold 2 elements  
    employees(1) := employee_type(101, 'John', 'Doe', 5000);  
    employees(2) := employee_type(102, 'Jane', 'Smith', 6000);  
  
    -- Loop through the collection  
    FOR i IN 1 .. employees.COUNT LOOP  
        DBMS_OUTPUT.PUT_LINE('Employee ID: ' ||  
employees(i).employee_id);  
        DBMS_OUTPUT.PUT_LINE('First Name: ' ||  
employees(i).first_name);  
        DBMS_OUTPUT.PUT_LINE('Last Name: ' ||  
employees(i).last_name);  
        DBMS_OUTPUT.PUT_LINE('Salary: ' || employees(i).salary);  
    END LOOP;  
END;  
/
```

Object Types & Collection Example(2/2)

Explanation of Key Parts

- **EXTEND**: Expands the size of the collection to accommodate additional elements.
- **COUNT**: Returns the number of elements in the collection.
- **Looping**: A FOR loop is used to iterate through each element in the collection.

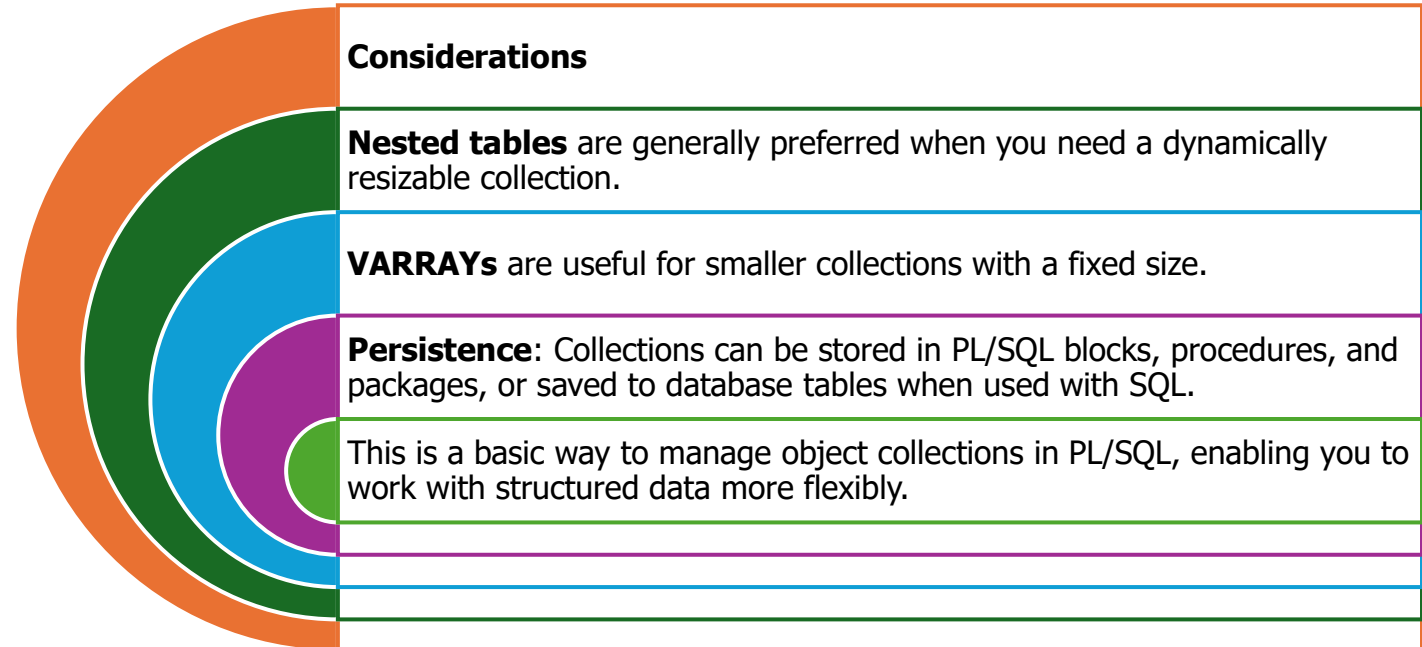
Example Using a VARRAY Collection

If you're using a VARRAY, the process is similar, but with a fixed limit:

```
DECLARE
    employees employee_varray := employee_varray(
        employee_type(101, 'Alice', 'Johnson', 4500),
        employee_type(102, 'Bob', 'Miller', 4800)
    );
BEGIN
    FOR i IN 1 .. employees.COUNT LOOP
        DBMS_OUTPUT.PUT_LINE('Employee ID: ' ||
            employees(i).employee_id);
        DBMS_OUTPUT.PUT_LINE('First Name: ' ||
            employees(i).first_name);
        DBMS_OUTPUT.PUT_LINE('Last Name: ' ||
            employees(i).last_name);
        DBMS_OUTPUT.PUT_LINE('Salary: ' || employees(i).salary);
    END LOOP;
END;
/
```

3. Using the Object Collection in PL/SQL

You can use these types in PL/SQL blocks to manipulate collections of objects. Here's an example that demonstrates creating an *employee_table*, adding elements, and iterating through the collection.



Object Types – Example (1/2)

1. Create an Object Type

```
-- Create an object type to represent an employee
CREATE OR REPLACE TYPE EmployeeType AS OBJECT (
--employee_id, first_name, last_name, and salary are attributes of the
object.
    employee_id    NUMBER,
    first_name     VARCHAR2(50),
    last_name      VARCHAR2(50),
    salary         NUMBER(10,2),
-- A MEMBER FUNCTION full_name is defined to return the full name of
the employee by combining first and last names.
    MEMBER FUNCTION full_name RETURN VARCHAR2
);
/
```

2. Implementing Object Methods

```
-- Implement the MEMBER FUNCTION full_name
CREATE OR REPLACE TYPE BODY EmployeeType AS
    MEMBER FUNCTION full_name RETURN VARCHAR2 IS
    BEGIN
        RETURN first_name || ' ' || last_name;
    END;
END;
/
```

3. Using Object Types in PL/SQL Blocks



```
DECLARE
    emp EmployeeType; -- Declare a variable of type EmployeeType
BEGIN
    -- Instantiate the object
    emp := EmployeeType(101, 'John', 'Doe', 5000.00);

    -- Access and modify object attributes
    DBMS_OUTPUT.PUT_LINE('Employee ID: ' || emp.employee_id);
    DBMS_OUTPUT.PUT_LINE('Employee Name: ' || emp.full_name);
    DBMS_OUTPUT.PUT_LINE('Employee Salary: ' || emp.salary);

    -- Update salary
    emp.salary := 5500.00;
    DBMS_OUTPUT.PUT_LINE('Updated Salary: ' || emp.salary);
END;
/
```

In this example:

- `emp` is an instance of *EmployeeType*.
- The attributes *employee_id*, *first_name*, *last_name*, and *salary* are accessed and updated.
- The *full_name* method is called to display the employee's full name.

Summary

Object types in PL/SQL allow you to encapsulate both data and behavior, which can be beneficial for complex data modeling.

Object Types – Example (2/2)

4. Using Object Types in Tables

```
-- Create a table to store employee objects
CREATE TABLE employees (
  emp_data EmployeeType
);
```

Example: Inserting and Querying Object Data in Tables

```
-- Insert an employee record
INSERT INTO employees VALUES (EmployeeType(102, 'Jane',
'Smith', 6000.00));

-- Query and display employee data
DECLARE
  emp EmployeeType;
BEGIN
  SELECT emp_data INTO emp FROM employees WHERE
emp_data.employee_id = 102;

  DBMS_OUTPUT.PUT_LINE('Employee Name: ' ||
emp.full_name);
  DBMS_OUTPUT.PUT_LINE('Employee Salary: ' || emp.salary);
END;
/
```

5. Using Collections of Object Types

You can also create collections (nested tables, VARRAYs) of object types, which is useful when you want to handle multiple records as a single entity.

```
-- Create a collection of EmployeeType objects
CREATE OR REPLACE TYPE EmployeeTable AS TABLE OF EmployeeType;
/
```

Example: Using Collection of Object Types in PL/SQL

```
DECLARE
  employees EmployeeTable := EmployeeTable(); -- Initialize an empty
collection
BEGIN
  -- Add employees to the collection
  employees.EXTEND(2); -- Extend the collection size
  employees(1) := EmployeeType(103, 'Alice', 'Brown', 4800.00);
  employees(2) := EmployeeType(104, 'Bob', 'White', 5200.00);

  -- Loop through the collection
  FOR i IN 1 .. employees.COUNT LOOP
    DBMS_OUTPUT.PUT_LINE('Employee Name: ' || employees(i).full_name);
    DBMS_OUTPUT.PUT_LINE('Employee Salary: ' || employees(i).salary);
  END LOOP;
END;
/
```

Object Type as a Record

An **object type** in Oracle can also act as a "record" in that it allows you to store and manage multiple fields of different types together. However, object types can also include methods, whereas records cannot.

Example: Using Object Type as a Record

Here's how an object type can serve a similar purpose as a record type, but with added functionality:

```
DECLARE
-- Define a custom record type for employee details
TYPE EmployeeRec IS RECORD (
    employee_id NUMBER,
    first_name  VARCHAR2(50),
    last_name   VARCHAR2(50),
    salary      NUMBER(10, 2)
);

-- Declare a variable of the EmployeeRec type
emp_rec EmployeeRec;
BEGIN
-- Assign values to each field in the record
emp_rec.employee_id := 101;
emp_rec.first_name  := 'John';
emp_rec.last_name   := 'Doe';
emp_rec.salary      := 5000.00;

-- Display employee details
DBMS_OUTPUT.PUT_LINE('Employee ID: ' || emp_rec.employee_id);
DBMS_OUTPUT.PUT_LINE('Employee Name: ' || emp_rec.first_name || ' ' ||
emp_rec.last_name);
DBMS_OUTPUT.PUT_LINE('Employee Salary: ' || emp_rec.salary);
END;
/
```

Creating a Function that Returns a Record

```
CREATE OR REPLACE FUNCTION get_employee_details(p_employee_id NUMBER)
RETURN VARCHAR2 IS
    -- Define a record type to hold employee data
    TYPE EmployeeRec IS RECORD (
        employee_id NUMBER,
        first_name VARCHAR2(50),
        last_name VARCHAR2(50),
        salary NUMBER(10, 2)
    );

    -- Declare a variable of the EmployeeRec type
    emp_rec EmployeeRec;

BEGIN
    -- Retrieve employee details from the employees table
    SELECT employee_id, first_name, last_name, salary
    INTO emp_rec
    FROM employees
    WHERE employee_id = p_employee_id;

    -- Display the retrieved employee details
    DBMS_OUTPUT.PUT_LINE('Employee ID: ' || emp_rec.employee_id);
    DBMS_OUTPUT.PUT_LINE('Employee Name: ' || emp_rec.first_name || ' ' || emp_rec.last_name);
    DBMS_OUTPUT.PUT_LINE('Employee Salary: ' || emp_rec.salary);

    -- Return a concatenated string with employee details
    RETURN 'ID: ' || emp_rec.employee_id ||
        ', Name: ' || emp_rec.first_name || ' ' || emp_rec.last_name ||
        ', Salary: ' || emp_rec.salary;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN 'Employee not found';
END get_employee_details;
/
```

In this function:

- We define a **record type** called EmployeeRec to hold the structure of the employee data (i.e., employee_id, first_name, last_name, and salary).
- We declare a variable emp_rec of type EmployeeRec.
- We use a SELECT INTO statement to populate emp_rec with data from the employees table, based on the p_employee_id parameter.
- The function then returns a concatenated string with the employee's details or an error message if the employee ID is not found.

In Oracle PL/SQL, a **record** is a data structure that allows you to store multiple fields of different data types under a single variable name, similar to a row of data in a table. Unlike object types, which can include both data and methods, records are simpler and typically used for temporary storage within PL/SQL blocks, especially for handling query results or creating custom structures without requiring database persistence.

Step 2: Test the Function

```
-- Test the function for an existing employee
DECLARE
    employee_info VARCHAR2(200);
BEGIN
    employee_info := get_employee_details(101);
    DBMS_OUTPUT.PUT_LINE(employee_info);
END;
/

-- Test the function for a non-existent employee
DECLARE
    employee_info VARCHAR2(200);
BEGIN
    employee_info := get_employee_details(999);
    DBMS_OUTPUT.PUT_LINE(employee_info);
END;
/
```

Summary

This example demonstrates:

- Defining a record type inside a function.
- Using a record variable to retrieve data from a table.
- Returning data from the function using the retrieved record details.

This structure is useful for creating modular code where you may want to retrieve and work with grouped data temporarily within PL/SQL, without creating complex persistent objects.

Step 2: Test the Function

Comparing Records and Object Types

Feature	PL/SQL Record	Object Type
Methods	Cannot have methods	Can include methods like full_name
Scope	Only within PL/SQL block/package	Schema-level, reusable across PL/SQL and database tables
Usage in Tables	Cannot be used as a column type	Can be used as a column in tables
Initialization	Direct assignment of fields	Instantiation using TypeName(...) syntax

Summary

- Use **PL/SQL records** when you need a simple, temporary structure within a PL/SQL block or package and don't need extra functionality or schema-wide access.
- Use **object types** when you want more flexibility, such as defining methods on the structure, using the structure across PL/SQL blocks, or storing it in database tables.

Both structures can be highly useful, depending on the complexity of the operations you need and whether the data needs to persist across sessions or be used in database tables.

A Guide to the AUTHID Clause in Oracle PL/SQL

In Oracle PL/SQL, **AUTHID** is a clause used in stored procedures, functions, or packages to specify the execution rights context for the PL/SQL code. The **AUTHID** clause determines whether the PL/SQL code runs with the privileges of the user who created (or owns) the code or the user who is executing the code. This feature allows you to control privilege delegation and security.



Options to Use AUTHID

When you create a PL/SQL procedure or function, there is an optional parameter **AUTHID** with 2 options as **[CURRENT USER]** or **[DEFINER]**, which could be used to distinguish between the owner and the user. By default, Oracle creates **[DEFINER]** object.

AUTHID Clause in Oracle PL/SQL Packages: DEFINER vs. CURRENT_USER



DEFINER



☐ When you specify **AUTHID DEFINER**, the code in the package or procedure executes with the privileges of the user who owns the package or procedure (the definer).

☐ This means that if the definer has certain privileges on database objects (like tables, views, etc.), those privileges are used when executing the code, regardless of who is calling it.

CURRENT USER



☐ If you specify **AUTHID CURRENT_USER**, the code executes with the privileges of the user **who is currently running the code (the invoker)**.

☐ This approach can be useful when you want to ensure that the code respects the caller's permissions.

Best Practices

Use AUTHID CURRENT_USER as the default unless you have a specific reason to use AUTHID DEFINER.

Grant privileges to roles rather than individual users for easier management.

Regularly review and audit your database security to ensure it aligns with your organization's policies.

PL/SQL Packages – AUTHID Clause Advantages



Advantages of AUTHID DEFINER

Security: By using AUTHID DEFINER, you can encapsulate sensitive operations. For instance, if a procedure needs access to a restricted table, you can grant that access to the user who owns the procedure without granting it to every user who calls the procedure.

Simplified Access Control: It allows for more straightforward management of permissions since the owner of the procedure is responsible for the underlying security of the operations.

Consistency: It ensures that the behavior of the procedure or package remains consistent regardless of who is calling it. This is particularly useful in multi-user environments.

Scenario 0: Managing User Privileges in Oracle PL/SQL: AUTHID Scenarios with Users A, B, and C (1/3)

In Oracle PL/SQL, the AUTHID clause specifies the privileges under which a stored procedure, function, or package runs. This is important in cases where the procedure is executed by multiple users, and you want to control whether it uses the caller's or the definer's privileges. Here's how it works, with AUTHID options DEFINER and CURRENT_USER.

AUTHID DEFINER

- When you use **AUTHID DEFINER (the default mode)**, the procedure or function runs with the privileges of the user who owns the procedure, function, or package.
- For instance, if User A creates a procedure with **AUTHID DEFINER**, it will run with User **A's privileges, regardless** of who calls it.

```
CREATE OR REPLACE PROCEDURE my_procedure
AUTHID DEFINER
AS
BEGIN
    -- Procedure code here
END;
```



- If User B calls my_procedure, Oracle checks User A's privileges for any objects accessed within my_procedure, not User B's.
- This is useful for security because it limits access to sensitive objects or operations that only the definer (User A) can perform.

Scenario 0: Managing User Privileges in Oracle PL/SQL: AUTHID Scenarios with Users A, B, and C (3/3)

Practical Example with Users A, B, and C

Suppose

- **User A** is the owner of a sensitive table and has a procedure to update it.
- **User B** and **User C** have execute privileges on this procedure but no direct access to the table.

With AUTHID DEFINER

- If the procedure is defined with **AUTHID DEFINER**, User B and User C can call the procedure, and it will use User A's privileges to update the table (even though B and C don't have direct table access).

With AUTHID CURRENT_USER

- If the procedure is defined with **AUTHID CURRENT_USER**, then when B or C calls the procedure, they need **to have direct privileges** on the table. The procedure will fail if they lack access rights because it uses the caller's privileges.

In summary:

- Use **AUTHID DEFINER** when the procedure should use the definer's (creator's) privileges, regardless of the caller.
- Use **AUTHID CURRENT_USER** when you want the procedure to rely on the caller's privileges, providing a more dynamic and adaptable access control based on the current user.



Scenario 1: Banking Application with Role-Based Access

Imagine a banking application where different roles, such as **Tellers**, **Managers**, and **Auditors**, have different privileges for accessing and modifying customer data. Here's how the AUTHID clause can be used effectively:

- **Problem:** The bank has procedures that should only allow certain users to perform specific actions. For instance, only a **Teller** should be able to create transactions, while a **Manager** can approve high-value transactions. An **Auditor** should be able to view, but not modify, any customer data.
- **Solution with AUTHID:**
 - Use **AUTHID DEFINER** for procedures that should always run with the privileges of the procedure's owner, such as **audit procedures**. For example, an audit log procedure could have **AUTHID DEFINER**, ensuring that anyone running it can capture audit logs even if they don't have privileges to view or alter customer data.
 - Use **AUTHID CURRENT_USER** for procedures that need to run with the privileges of the caller, allowing them to work within their own privilege limitations. For example, a **create transaction procedure** can be defined with **AUTHID CURRENT_USER** to restrict actions based on the privileges of the teller or manager executing it.

This setup ensures that the application enforces role-based access control while keeping security intact.



Scenario 2: Healthcare System with Multi-Level Access

In a healthcare system, different types of users, such as **Doctors**, **Nurses**, and **Administrators**, have different privileges over patient data.

- **Problem:** Only **Doctors** should be able to view and update patient diagnosis data, **Nurses** should only be able to update basic patient information, and **Administrators** should have access to all information but not directly modify any medical data.
- **Solution with AUTHID:**
 - Procedures used by **Doctors** could be created with **AUTHID DEFINER**, ensuring they run with the privileges of the owner (e.g., a central “healthcare application” user) and allow only authorized medical personnel to update critical information.
 - Other procedures, such as updating general patient information, could use **AUTHID CURRENT_USER**, ensuring that each user’s access is based on their actual privileges in the system.

Using AUTHID this way ensures the system can verify user access at a granular level, allowing different users to work with the same set of procedures securely and in accordance with their role-based permissions.



Scenario 3: Multi-Tenant SaaS Application

In a Software-as-a-Service (SaaS) application that serves multiple organizations (tenants), each organization's data must be isolated and accessible only by users from that organization.

- **Problem:** Procedures that access data should be restricted to data that belongs only to the user's organization. For example, `get_employee_records` should only return employees from the user's organization, not other organizations.
- **Solution with AUTHID:**
 - Using `AUTHID CURRENT_USER` in procedures that access sensitive or tenant-specific data allows the code to dynamically adapt to the privileges of each user or organization.
 - The database can enforce additional security checks based on the caller's privileges, making sure users can only access their own organization's data.
 - Procedures with broader access, such as global configurations, could still use `AUTHID DEFINER` to ensure they run with a higher level of privilege.

By combining `AUTHID DEFINER` and `AUTHID CURRENT_USER`, the SaaS application can isolate tenant data while still allowing flexible access control.



In each of these scenarios, the `AUTHID` clause allows developers to control how privileges are applied to stored procedures and packages, ensuring secure and context-specific access in multi-user and multi-role environments.



Sample Code - PL/SQL Procedure (Named Block) & PL/SQL Function

```
-- Create a Proc with AUTHID Option
CREATE OR REPLACE PROCEDURE myProc (i_Directory IN
VARCHAR2)
AUTHID DEFINER | CURRENT_USER
AS
-- procedure code
BEGIN
-- statements
END myProc;
/
```

```
-- Create a Function with AUTHID Option
CREATE OR REPLACE FUNCTION myFunc (i_Directory IN VARCHAR2)
AUTHID DEFINER | CURRENT_USER
AS
-- Function code
BEGIN
-- statements
END myFunc;
/
```

- **AUTHID DEFINER:** The procedure runs with the privileges of the user who created it (the "definer"). This is the default if you don't specify an AUTHID.
- **AUTHID CURRENT_USER:** The procedure runs with the privileges of the user who calls it (the "invoker" or current user).



Sample Code - PL/SQL Procedure (Named Block) & PL/SQL Function

```
-- Example: Create a Procedure with AUTHID Option
CREATE OR REPLACE PROCEDURE myProc (i_Directory IN VARCHAR2)
AUTHID DEFINER -- or AUTHID CURRENT_USER
AS
    -- Declare any variables if needed
BEGIN
    -- Add procedure code, for example:
    DBMS_OUTPUT.PUT_LINE('Processing directory: ' || i_Directory);

    -- More statements or operations as needed
END myProc;
/
```

Explanation

- **AUTHID DEFINER:** This option makes the procedure execute with the privileges of the user who created it (the definer).
- **AUTHID CURRENT_USER:** This option makes the procedure execute with the privileges of the user who is calling it (the invoker).

Replace **AUTHID DEFINER** or **AUTHID CURRENT_USER** with the option that fits your requirements.

```
-- Create or replace a procedure named 'myProc' that takes a directory
name as input
CREATE OR REPLACE PROCEDURE myProc (i_Directory IN VARCHAR2)
AUTHID CURRENT_USER -- Set execution privileges to the current user
(invoker rights)

AS
BEGIN
    -- Output a message to the console with the directory name provided
    by the user
    DBMS_OUTPUT.PUT_LINE('Accessing directory: ' || i_Directory);

    -- Additional statements or logic could go here if needed
END myProc;
/ -- End of procedure definition
```

AUTHID CURRENT_USE

PL/SQL Packages – AUTHID Clause Example I

Example

In this example:

- If my_package is created with AUTHID DEFINER, the user who owns the package must have privileges to insert into my_table.
- Even if some_user (who does not have insert privileges on my_table) calls insert_value, it will succeed because the execution uses the privileges of the package owner.

Summary

- **AUTHID DEFINER** allows a PL/SQL package or procedure to execute with the privileges of the user who defined it, regardless of who is calling it. This can enhance security and simplify permission management in multi-user databases.
- Conversely, **AUTHID CURRENT_USER** executes with the privileges of the user calling the procedure, which is more flexible but may require careful management of permissions.

Conclusion

The AUTHID clause is a valuable tool for implementing robust security measures in Oracle databases. By understanding its different settings and how to use them effectively, you can ensure that your data is protected and accessed only by authorized users. Remember to choose the appropriate AUTHID setting based on your specific requirements and security considerations.

```
-- Create a table for demonstration
CREATE TABLE my_table (
  id NUMBER PRIMARY KEY,
  value VARCHAR2(100)
);

-- Grant SELECT privilege to a specific user
GRANT SELECT ON my_table TO some_user;

-- Create a package with AUTHID DEFINER
CREATE OR REPLACE PACKAGE my_package AUTHID DEFINER IS
  PROCEDURE insert_value(p_id NUMBER, p_value VARCHAR2);
END my_package;
/

-- Create the package body
CREATE OR REPLACE PACKAGE BODY my_package AS
  PROCEDURE insert_value(p_id NUMBER, p_value VARCHAR2) IS
  BEGIN
    -- The procedure inserts a value into the table
    INSERT INTO my_table (id, value) VALUES (p_id, p_value);
  END insert_value;
END my_package;
/

-- Assume 'some_user' calls the procedure
BEGIN
  my_package.insert_value(1, 'Hello, World!');
END;
```

PL/SQL Packages – AUTHID Clause Example II



Package & Procedure

```
-- Create a package named 'aa_pkg'
CREATE OR REPLACE PACKAGE aa_pkg AUTHID DEFINER IS
  -- Declare an associative array type 'aa_type'
  -- The array is indexed by VARCHAR2(15) and stores INTEGER values
  TYPE aa_type IS TABLE OF INTEGER INDEX BY VARCHAR2(15);
END;
/

-- Create a procedure 'print_aa' that takes an associative array of type
'aa_pkg.aa_type' as a parameter
CREATE OR REPLACE PROCEDURE print_aa (
  aa aa_pkg.aa_type -- Parameter 'aa' of type 'aa_pkg.aa_type' (associative array)
) AUTHID DEFINER IS
  -- Declare a variable 'i' of type VARCHAR2(15)
  -- This will hold the current key in the associative array during iteration
  i VARCHAR2(15);
BEGIN
  -- Initialize 'i' with the first key in the associative array
  i := aa.FIRST;

  -- Loop through the associative array while the key 'i' is not NULL
  WHILE i IS NOT NULL LOOP
    -- Print the value corresponding to the key 'i' and the key itself using
    DBMS_OUTPUT
      DBMS_OUTPUT.PUT_LINE (aa(i) || ' ' || i);

    -- Get the next key in the associative array
    i := aa.NEXT(i);
  END LOOP; -- End of the loop
END;
/
```

```
-- Declare a block to test the package and procedure
DECLARE
  -- Declare a variable 'aa_var' of the associative array type
  defined in the 'aa_pkg' package
  aa_var aa_pkg.aa_type;
BEGIN
  -- Assign values to the associative array using string keys
  ('zero', 'one', 'two')
  aa_var('zero') := 0;
  aa_var('one') := 1;
  aa_var('two') := 2;

  -- Call the 'print_aa' procedure to output the associative array
  values and keys
  print_aa(aa_var);
END;
/
```

Test Package and Procedure

PL/SQL Packages – Example

Step 1: Create the Package Specification

This is where we define the interface (i.e., what functions and procedures are available publicly).

```
CREATE OR REPLACE PACKAGE pattern_package AS
  -- Procedure to draw a normal triangle pattern
  PROCEDURE draw_triangle(p_rows IN NUMBER);

  -- Procedure to draw an inverted triangle pattern
  PROCEDURE draw_inverted_triangle(p_rows IN NUMBER);
END pattern_package;
/
```

Step 2: Create the Package Body

This is where the actual logic of the package (procedures) is implemented.

CREATE OR REPLACE PACKAGE BODY pattern_package AS

```
  -- Procedure to draw a normal triangle pattern
  PROCEDURE draw_triangle(p_rows IN NUMBER) IS
  BEGIN
    FOR i IN 1..p_rows LOOP
      FOR j IN 1..i LOOP
        DBMS_OUTPUT.PUT('*');
      END LOOP;
      DBMS_OUTPUT.PUT_LINE("");
    END LOOP;
  END draw_triangle;
```

-- Procedure to draw an inverted triangle pattern

```
PROCEDURE draw_inverted_triangle(p_rows IN NUMBER) IS
BEGIN
  FOR i IN REVERSE 1..p_rows LOOP
    FOR j IN 1..i LOOP
      DBMS_OUTPUT.PUT('*');
    END LOOP;
    DBMS_OUTPUT.PUT_LINE("");
  END LOOP;
END draw_inverted_triangle;
END pattern_package;
/
```

Step 3: Calling the Procedures from the Package

Once the package is created, you can call its procedures in PL/SQL blocks or SQL scripts. Here's an example of how to call these procedures:

Example 2: Draw an Inverted Triangle

```
BEGIN
  pattern_package.draw_inverted_triangle(5); -- Drawing an inverted triangle with 5 rows
END;
/
```

Example 2: Draw an Inverted Triangle

```
BEGIN
  pattern_package.draw_inverted_triangle(5); -- Drawing an inverted triangle with 5 rows
END;
/
```

PL/SQL Packages – Explanation:

Package Specification (my_package)

This is the interface where we declare the functions and procedures that can be accessed by other PL/SQL blocks.



Package Body

This contains the actual implementation of the logic for each function or procedure.



Performance

Once a package is loaded into memory, subsequent calls to its procedures or functions are faster.

Oracle Pipelined Table Functions

Overview

Pipelined table functions allow you to return data as a table (or set of rows) from a PL/SQL function. Unlike regular functions that return a single value or collection, pipelined functions return rows one at a time and can be queried like a regular table.

What is a Data Pipeline?

A **data pipeline** is a series of processes used to move and transform data from one system to another, enabling businesses to efficiently integrate data for analytics and operational use. These processes include **ingesting**, **processing**, **transforming**, and **enriching** data, whether structured, unstructured, or semi-structured.

Benefits

Performance: Rows can be processed and returned to the caller immediately rather than waiting for the entire dataset to be generated.

Scalability: They can efficiently handle large data sets by returning rows as they are processed.

Flexibility: They can be integrated easily into SQL statements, allowing for more complex queries.

Oracle Pipelined – Diagram (1/2)



Source Image: <https://vocal.media/01/oracle-cloud-pipelines>

Diagram Overview:

The image illustrates the typical flow of data from **source systems** (Salesforce, Oracle Enterprise Resource Planning (ERP) Cloud, Workday) through an ETL (Extract Transform Load) process, into a **data warehouse**, and finally into **reporting and analytics**.

Explanation Using Oracle Pipelined Table Functions:

1. Source Systems (Salesforce, Oracle ERP Cloud, Workday)

•These represent various **data sources** from which the raw data is extracted. In the context of Oracle, data from these sources can be read into the database using **External Tables** or **SQL*Loader**.

2. ETL/ELT Process

•This is where Oracle **pipelined table functions** can come into play. A **pipelined table function** allows data to be processed in a **streaming fashion** without requiring intermediate storage or staging tables. This can increase efficiency, especially for **real-time** or **large-scale processing** tasks.

- **Extract:** Data is extracted from the source systems.
- **Transform:** The transformation logic (such as cleansing, filtering, or enriching data) is applied directly through the pipelined table function. This allows the data to be transformed **on-the-fly** as it moves through the pipeline.
- **Load:** The transformed data can be immediately loaded into the **data warehouse**, without the need for a separate staging area. Oracle pipelined functions can output rows directly to the target.



Oracle Pipelined – Diagram (2/2)

3. Data Warehouse

- After the data is processed by the pipelined table function, it is loaded into the **data warehouse**, where it is stored for **analytics and reporting** purposes. The pipelined function's ability to process rows in a continuous stream means that it can handle large volumes of data efficiently, perfect for scenarios like feeding a warehouse in real-time or near real-time.

4. Reporting and Analytics

- Finally, the processed and transformed data from the data warehouse is used for **business intelligence, analytics, or dashboarding**. Since pipelined table functions can handle data in **real-time**, they are ideal for situations where you need **up-to-date** or **dynamic reporting**.

Key Benefits of Using Pipelined Table Functions in This Process:

- **Efficiency:** Pipelined functions process data row by row, avoiding the need for intermediate storage like a staging table.
- **Performance:** They can handle large datasets more efficiently, as the data is processed as it is produced.
- **Parallelization:** These functions can be parallelized, which means you can process large volumes of data in a shorter amount of time.⁴

Example of Pipelined Table Function Use:

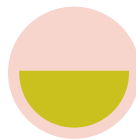
In an Oracle pipeline, you might extract data from **Salesforce** (via a connector or external table), transform the data with a pipelined function (e.g., filter out invalid records or apply calculations), and then load the results directly into your **data warehouse** for analytics, skipping unnecessary intermediate steps.

In summary, Oracle pipelined table functions offer a streamlined, high-performance method of handling the **ETL process**, particularly when real-time or large-scale data processing is required. This makes them a great fit for the type of data flow shown in the diagram.

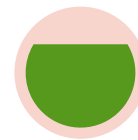
Optimizing Large-Scale Data Processing with Oracle Pipelined Table Functions: When and Why to Use Them



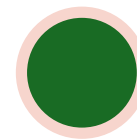
When to Use Pipelined Table Functions



When you need to process large datasets row by row.



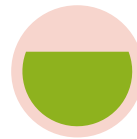
When encapsulating complex PL/SQL logic and making it available to SQL queries.



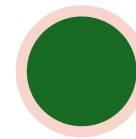
When you want to stream data for processing rather than waiting for an entire dataset to be returned.



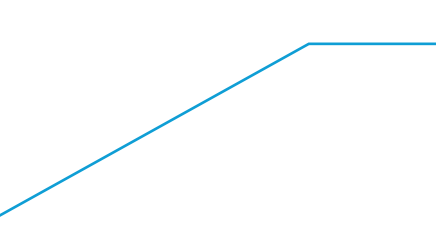
Drawbacks



Pipelined table functions can be more complex to implement than regular SQL or PL/SQL solutions.



In some cases, especially for smaller datasets, the performance gains may be minimal, and the added complexity might not be worth it.

A blue line graphic consisting of a diagonal line segment followed by a horizontal line segment, ending at a vertical line.

Pipelined table functions are a great tool when you need high performance, memory-efficient processing of large datasets, especially in cases where the data is produced or consumed in stages.

Oracle Regular & Pipelined Table Functions - Syntax

```
-- Step 1: Create the object type (if not already created)
CREATE OR REPLACE TYPE object_type_name AS OBJECT (
    column1_name datatype1,
    column2_name datatype2,
    ...
);
/

-- Step 2: Create the collection type (if not already created)
CREATE OR REPLACE TYPE collection_type_name AS TABLE OF
object_type_name;
/

-- Step 3: Create the pipeline table function
CREATE OR REPLACE FUNCTION function_name (
    parameter1 datatype,
    parameter2 datatype
) RETURN collection_type_name PIPELINED IS
BEGIN
    -- Loop through the result set and pipe rows
    FOR record IN (SELECT ... FROM ... WHERE ...) LOOP
        PIPE ROW(object_type_name(record.column1, record.column2, ...));
    END LOOP;

    RETURN; -- Implicitly return at the end
END function_name;
/
```

Reason for defining types in Oracle PL/SQL:



1. Structured Data Representation:

- Encapsulates related data into a single logical unit, enhancing code readability.

2. Code Reusability:

- Reusable across multiple programs, reducing redundancy and ensuring consistency.

3. Encapsulation:

- Combines data and behavior, enforcing data integrity and business rules.

4. Complex Data Handling:

- Supports collections (arrays, nested tables) for easier management of sets of data and enables bulk operations for performance optimization.

5. Type Safety:

- Ensures type safety with compile-time checks, catching errors early.

6. Enhanced Performance:

1. Can lead to performance improvements, especially with bulk operations.

7. Interfacing with External Systems:

- Facilitates integration with other applications, making it easier to pass structured data.

In summary, defining types improves organization, modularity, efficiency, and maintainability in database programming. If you need any more information or examples, just let me know!

Oracle Regular & Pipelined Table Functions – Example I

To create a pipeline table function in Oracle, you'll need to follow a specific syntax and structure. Pipeline table functions allow you to return a set of rows (a collection) that can be queried like a regular table. This is particularly useful when dealing with large datasets, as it enables row-by-row processing while still allowing the use of SQL.

Example Scenario Let's create a pipeline table function that retrieves allowances for a given employee. The function will return a table with allowance details such as ALLOWANCE_ID, ALLOWANCE_AMOUNT, and EFFECTIVE_DATE.

Step 1: Define an Object Type

First, we need to define an object type that represents the structure of the data we want to return from the pipeline function.

```
-- Create or replace an object type named allowance_record
CREATE OR REPLACE TYPE allowance_record AS OBJECT (
  -- Define a property for the effective id, amount and date of the allowance
  ALLOWANCE_ID NUMBER,
  ALLOWANCE_AMOUNT NUMBER,
  EFFECTIVE_DATE DATE
);
/
```

Step 2: Create Table Type

```
CREATE OR REPLACE TYPE allowance_table AS TABLE OF allowance_record;
/
```

Step 3: Create Pipelined Function

-- Function to retrieve all allowances for a given employee
-- Accepts an employee ID as input and returns a pipelined table of allowances

```
CREATE OR REPLACE FUNCTION get_employee_allowances(emp_id IN NUMBER)
RETURN allowance_table PIPELINED -- Specifies that this function returns a pipelined table type
AS
BEGIN
  -- Loop through each record in the employee_allowances table
  FOR rec IN (
    SELECT allowance_id, allowance_amount, effective_date
    FROM employee_allowances
    WHERE employee_id = emp_id -- Only select records matching the input employee ID
  )
  LOOP
    -- Pipe each row of data into the allowance_table pipeline
    PIPE ROW(allowance_record(rec.allowance_id, rec.allowance_amount, rec.effective_date));
  END LOOP;

  RETURN; -- End of function; return control to the caller
END;
/
```

Step 2: Define a Table Type

Next, we define a collection type to hold multiple allowance_record objects.

```
-- Create or replace a collection type named allowance_table
CREATE OR REPLACE TYPE allowance_table AS TABLE OF allowance_record;
/
```

Summary

- **Object Type:** Defines the structure of individual records.
- **Table Type:** Holds a collection of those records.
- **Pipelined Function:** Processes and returns rows incrementally using the

Oracle Regular & Pipelined Table Functions – Example II

1. Creating the Object Type:

Defines the structure of the data you want to return. Each field in the object corresponds to a column in the output.

```
CREATE OR REPLACE TYPE allowance_record AS OBJECT (  
    ALLOWANCE_ID NUMBER,  
    ALLOWANCE_AMOUNT NUMBER,  
    EFFECTIVE_DATE DATE  
);
```

2. Creating the Collection Type:

Defines a collection type that will hold multiple instances of the object type.

```
CREATE OR REPLACE TYPE allowance_table AS TABLE OF allowance_record;
```

4. Creating the Pipeline Table Function:

Defines the function that will return the collection type.

Inside the function, you select data from your tables and use a cursor loop to pipe each record into the output.

```
CREATE OR REPLACE FUNCTION get_employee_allowances (  
    p_employee_id NUMBER -- Parameter to accept the employee ID  
) RETURN allowance_table PIPELINED IS  
BEGIN  
    -- Loop through the allowances for the specified employee  
    FOR r IN (  
        SELECT a.ALLOWANCE_ID, a.ALLOWANCE_AMOUNT, a.EFFECTIVE_DATE  
        FROM ALLOWANCES a  
        JOIN EMPLOYEES e ON a.ROLE_ID = e.ROLE_ID -- Joining ALLOWANCES with EMPLOYEES using ROLE_ID  
        WHERE e.EMPLOYEE_ID = p_employee_id -- Filtering by the provided employee ID  
        AND a.IS_APPLICABLE = 'Y' -- Only consider applicable allowances  
        AND a.EFFECTIVE_DATE <= SYSDATE -- Only include allowances that are currently effective  
    ) LOOP  
        -- Pipe each allowance record into the output collection  
        PIPE ROW(allowance_record(r.ALLOWANCE_ID, r.ALLOWANCE_AMOUNT, r.EFFECTIVE_DATE));  
    END LOOP;  
  
    RETURN; -- Implicitly return at the end of the function  
END get_employee_allowances;  
/
```

Key Features of the Function

- 1. Parameter:** p_employee_id is used to filter allowances specific to a given employee.
- 2. Join Operation:** The function joins the ALLOWANCES table with the EMPLOYEES table on ROLE_ID, which allows us to filter allowances based on the employee's role.
- 3. Filtering Conditions:**
 - a.IS_APPLICABLE = 'Y': This condition ensures only applicable allowances are retrieved.
 - a.EFFECTIVE_DATE <= SYSDATE: This condition ensures only currently effective allowances are considered.
- 4. Piping Rows:** The function pipes rows from the result set into the output using PIPE ROW.

Querying the Pipeline Function

You can then use the TABLE() operator to query the results from your pipeline function:

```
SELECT *  
FROM TABLE(get_employee_allowances(101)); -- Replace '101' with the desired employee ID
```

Summary

This syntax outlines how to create a pipeline table function in Oracle PL/SQL, which can be a powerful tool for returning sets of data efficiently. If you need more specific examples or have any

Application of Procedure and Function

Introduction to Stored Procedures and Functions

- **Stored Procedure:** A block of SQL code stored on the database server, executed with a specific task in mind, and often used for batch processing or modular programming.
- **Function:** Similar to a procedure but must return a single value. Functions are ideal for computations or retrieving single data points.

Differences between Procedures and Functions

- **Return Type:** Functions must return a value; procedures don't have to.
- **Usage:** Functions can be called in SQL statements; procedures are typically called from PL/SQL blocks.
- **Execution Context:** Functions are often used for calculations; procedures are better suited for executing complex operations.

Advantages of Using Procedures and Functions

- **Modularity:** Code can be divided into manageable parts.
- **Reusability:** Reuse code across applications, improving consistency.
- **Performance:** Stored in compiled form, they execute faster.
- **Security:** Access to data can be controlled by restricting direct access to tables.
- **Error Handling:** Simplifies troubleshooting and improves reliability.

Application of Functions – Data Validation

Procedures can be used to **enforce data integrity** by ensuring that the data being entered into the database meets certain business rules or constraints.

Example: Validating User Input Before Insert

A procedure can validate the data before inserting it into a table. For example, if we want to make sure that employee salaries are always above a minimum threshold:

```
CREATE OR REPLACE PROCEDURE validate_salary(p_salary NUMBER) AS
BEGIN
    IF p_salary < 1000 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Salary cannot be less than 1000.');
```

Usage and Best Practices

Use functions for calculations or when a value needs to be returned.

Use procedures for multi-step operations that may not need to return a value.

Consider modularizing complex code with multiple functions and procedures.

Maintain readability by adding comments and appropriate error-handling.

This lecture equips you with a foundation to use stored procedures and functions effectively in Oracle, optimizing your code for performance and maintainability.

Conclusion: Key Takeaways – Solution

This is how you might implement this in a PL/SQL package that includes the procedure to update employee salaries and bonuses:

```
-- Create the package specification
CREATE OR REPLACE PACKAGE employee_management_pkg AS
    FUNCTION calculate_bonus(salary IN NUMBER) RETURN NUMBER; -- Function parameter
    PROCEDURE adjust_employee_salary(emp_id IN NUMBER, salary IN OUT NUMBER, new_salary OUT
NUMBER); -- Procedure parameters
END employee_management_pkg;
/

-- Create the package body
CREATE OR REPLACE PACKAGE BODY employee_management_pkg AS
    -- Function to calculate bonus (10% of salary)
    FUNCTION calculate_bonus(salary IN NUMBER) RETURN NUMBER IS
    BEGIN
        RETURN salary * 0.1; -- 10% bonus
    END calculate_bonus;

    -- Procedure to adjust the employee's salary and bonus
    PROCEDURE adjust_employee_salary(emp_id IN NUMBER, salary IN OUT NUMBER, new_salary OUT
NUMBER) IS
        v_bonus NUMBER; -- Variable to hold calculated bonus
    BEGIN
        -- Check if the employee exists and fetch the current salary
        SELECT SALARY INTO salary
        FROM EMPLOYEES
        WHERE EMPLOYEE_ID = emp_id;

        -- Logic for adjusting salary
        IF salary < 50000 THEN
            v_bonus := calculate_bonus(salary); -- Calculate bonus
            salary := salary + v_bonus; -- Increase salary by bonus
        ELSE
            salary := salary + 5000; -- Fixed increment for high earners
        END IF;

        new_salary := salary; -- Assign updated salary

        -- Update the employee's salary in the database
        UPDATE EMPLOYEES SET SALARY = salary WHERE EMPLOYEE_ID = emp_id;

        DBMS_OUTPUT.PUT_LINE('Updated salary for employee ID ' || emp_id || ' is ' || new_salary);

    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            DBMS_OUTPUT.PUT_LINE('Employee with ID ' || emp_id || ' not found.');
```

```
        WHEN OTHERS THEN
            DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
        END adjust_employee_salary;
    END employee_management_pkg;
/
```

Conclusion: Key Takeaways – Problem (1/4)



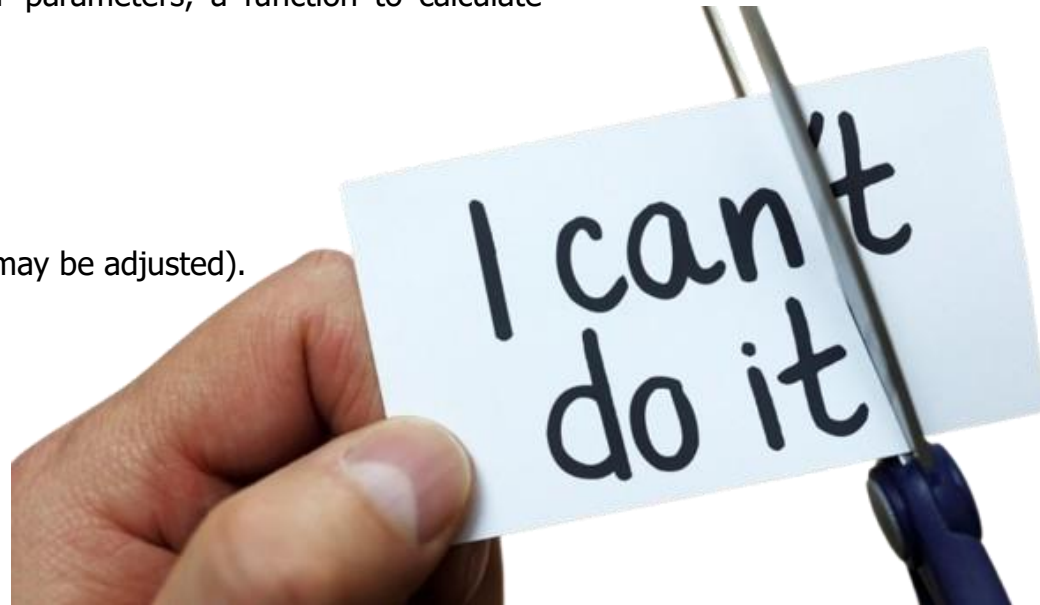
Based on the structure of the EMPLOYEES table provided, we can design a PL/SQL procedure that uses this table to manage employee salaries and bonuses. Here's how you could structure the procedure, incorporating the elements discussed earlier: IN, OUT, and INOUT parameters, a function to calculate bonuses, and the necessary logic to update the employee records.

Procedure Outline

1. Parameters:

- **IN:** emp_id (the ID of the employee).
- **INOUT:** salary (the current salary of the employee, which may be adjusted).
- **OUT:** new_salary (the updated salary after adjustments).

2. Function: To calculate the bonus based on the salary.



Conclusion: Key Takeaways – Problem (2/4)

I. Understanding Parameters:



1. Create a procedure named `adjust_employee_salary` that takes the following parameters:

- An IN parameter for `emp_id` (the ID of the employee).
- An INOUT parameter for `salary` (the current salary of the employee), which will be adjusted based on a specified bonus percentage.
- An OUT parameter for `new_salary` (the updated salary after adjustment).
- **Follow-up:** What will happen if the salary INOUT parameter is not assigned a value before the procedure ends? Explain.

II. Using Functions:

2. Create a function named `calculate_bonus` that accepts a NUMBER parameter representing the salary and returns a NUMBER representing the calculated bonus (e.g., 10% of the salary).

- **Follow-up:** How can you call this `calculate_bonus` function within your `adjust_employee_salary` procedure? Write the relevant part of the code.

III. Implementing Logic:

3. In your `adjust_employee_salary` procedure, implement the following logic:

- If the salary is below a certain threshold (e.g., \$50,000), increase it by the calculated bonus.
- If the salary is \$50,000 or above, increase it by a fixed amount (e.g., \$5,000).
- **Follow-up:** How would you handle cases where the salary exceeds a maximum limit after adjustment? Provide a conditional statement.

Conclusion: Key Takeaways – Problem (3/4)

IV. Package Creation:

4. Create a package named `employee_management_pkg` that includes your `adjust_employee_salary` procedure and the `calculate_bonus` function.
 - **Follow-up:** What are the components of the package specification and package body? Describe how to declare your procedures and functions in each.

V. Testing and Validation:

5. Write a test block to call the `adjust_employee_salary` procedure, passing an employee ID and a salary. Print the new salary after adjustment.
 - **Follow-up:** How would you modify the test block to handle potential errors, such as providing an invalid employee ID?

VI. Error Handling:

6. Integrate error handling in your `adjust_employee_salary` procedure to manage possible exceptions during the salary update, such as a non-existent employee ID.
 - **Follow-up:** What types of exceptions would be appropriate to catch, and how would you handle them?



Collaboration is key to learning! Work together with your group on these exercises, leveraging each member's strengths and perspectives. Make sure to communicate openly, share your insights, and help each other catch up on any concepts that might be unclear. Remember, the goal is not just to complete the exercises but to enhance your understanding collectively. Enjoy the process, and don't hesitate to ask questions along the way!

Conclusion: Key Takeaways – Problem (4/4)

Question:

Create a pipelined function that returns information about products and their pricing. Your task is to write a function named `get_product_pricing` that accepts a product category ID as input and returns a pipelined table containing the product ID, product name, and product price for each product in that category.



Steps:

1. Create a table named `products` with the following columns:
 - `product_id` (NUMBER): Primary key, representing the product ID.
 - `product_name` (VARCHAR2(100)): Name of the product.
 - `category_id` (NUMBER): Represents the category ID of the product.
 - `price` (NUMBER): Price of the product.
2. Insert sample data into the `products` table for at least three categories, each with several products.
3. Define an object type named `product_pricing_record` with the following attributes:
 - `product_id` (NUMBER)
 - `product_name` (VARCHAR2(100))
 - `price` (NUMBER)
4. Create a table type named `product_pricing_table` based on `product_pricing_record` to store multiple rows of `product_pricing_record`.
5. Write the `get_product_pricing` function to:
 - Accept a `category_id` as an input parameter.
 - Return a pipelined table of type `product_pricing_table`.
 - Loop through each product in the specified category and return each row in the pipeline with the product's ID, name, and price.

Conclusion: Key Takeaways – Solution

This is how you might implement this in a PL/SQL package that includes the procedure to update employee salaries and bonuses:

```
-- Create the package specification
CREATE OR REPLACE PACKAGE employee_management_pkg AS
    FUNCTION calculate_bonus(salary IN NUMBER) RETURN NUMBER; -- Function parameter
    PROCEDURE adjust_employee_salary(emp_id IN NUMBER, salary IN OUT NUMBER, new_salary OUT
NUMBER); -- Procedure parameters
END employee_management_pkg;
/

-- Create the package body
CREATE OR REPLACE PACKAGE BODY employee_management_pkg AS
    -- Function to calculate bonus (10% of salary)
    FUNCTION calculate_bonus(salary IN NUMBER) RETURN NUMBER IS
    BEGIN
        RETURN salary * 0.1; -- 10% bonus
    END calculate_bonus;

    -- Procedure to adjust the employee's salary and bonus
    PROCEDURE adjust_employee_salary(emp_id IN NUMBER, salary IN OUT NUMBER, new_salary OUT
NUMBER) IS
        v_bonus NUMBER; -- Variable to hold calculated bonus
    BEGIN
        -- Check if the employee exists and fetch the current salary
        SELECT SALARY INTO salary
        FROM EMPLOYEES
        WHERE EMPLOYEE_ID = emp_id;

        -- Logic for adjusting salary
        IF salary < 50000 THEN
            v_bonus := calculate_bonus(salary); -- Calculate bonus
            salary := salary + v_bonus; -- Increase salary by bonus
        ELSE
            salary := salary + 5000; -- Fixed increment for high earners
        END IF;

        new_salary := salary; -- Assign updated salary

        -- Update the employee's salary in the database
        UPDATE EMPLOYEES SET SALARY = salary WHERE EMPLOYEE_ID = emp_id;

        DBMS_OUTPUT.PUT_LINE('Updated salary for employee ID ' || emp_id || ' is ' || new_salary);

    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            DBMS_OUTPUT.PUT_LINE('Employee with ID ' || emp_id || ' not found. ');
        WHEN OTHERS THEN
            DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
    END adjust_employee_salary;
END employee_management_pkg;
/
```

Conclusion: Key Takeaways – Testing

Once you've corrected the package and compiled it successfully, you can test the procedure as follows:

```
DECLARE
  v_salary NUMBER;    -- Variable for current salary
  v_new_salary NUMBER; -- Variable for new salary after adjustment
BEGIN
  v_salary := 45000; -- Example salary
  employee_management_pkg.adjust_employee_salary(101, v_salary, v_new_salary); -- Adjusting salary for employee ID 101
END;
/
```



Keep Learning!